

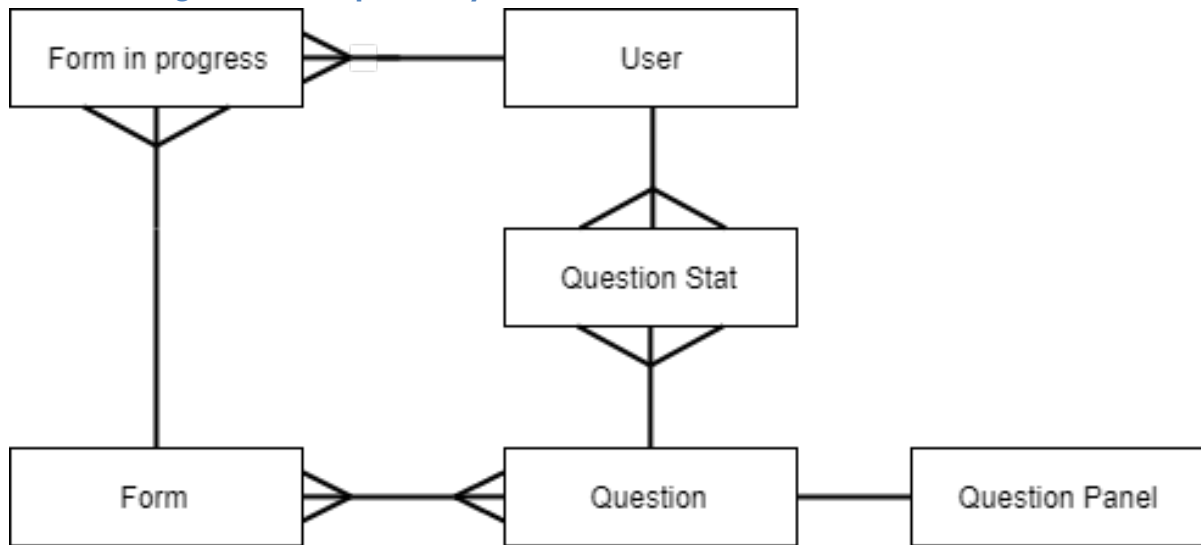
3 – Design

Contents

3.1	Breakdown of Problem	2
3.1.1	ER Diagram for Proposed System	2
3.1.2	Justification of Breakdown	2
3.1.3	Links between Objectives and Sub Problems	4
3.1.4	Sub-Problem Diagram	6
3.2	Input and Output Designs	7
3.2.1	Identification of Data	7
3.2.2	Designs of Input & Output	14
3.3	Files & Data Structures	43
3.3.1	Methods of Access inc. Files	43
3.3.3	Data Structure Designs	52
	Form.java	52
	Question.java	52
	QuestionPanel.java	52
	User.java	52
	QuestionStat.java	53
	FormInProgress.java	53
3.3.4	File Designs	54
3.4	Validation	55
3.5	Processes	57
3.5.1	Objectives Links with Data/Files	57
3.5.2	Processing Routines	65

3.1 Breakdown of Problem

3.11 ER Diagram for Proposed System



3.12 Justification of Breakdown

These are the main entities that are used in the system. This ER diagram shows how each should relate the other ones.

When a user attempts a question for the first time a question stat object will be created to store data about their attempts of the question. It will store their total number of times that they've failed the question's validation check, the most recent lengths of time that it's taken the user to complete the question and the number of times that it takes them to correct the error when they get the question wrong. The question stat stores the id of the question it relates to as a foreign key.

Each user needs multiple question stats because there are multiple questions that they can attempt. Each user has their own unique question stat database. Each question stat only has one user because it only contains the statistics of one users attempt. Each question stat only relates to one question. Each question may have multiple question stats (one for each user).

The question panel is the graphical element of a question and so each question relates to one and only one question panel. The question panel is linked to the question by storing the question's id as a foreign key. The question panel stores a list of the components that are in the question e.g. the labels and text fields.

A form is composed of multiple questions and a question may be in any number of forms and a form many have any number of questions. The form stores a list of the ids of the questions that make up it in order to form links.

The user can attempt a form in the system. When the form is attempted a copy of all of the question panels that are in the form is made so that the user can attempt the form without altering the 'master' question panels. The copies of the question panels are stored in a formInProgress object. Each form in progress relates to a single form because each form in progress stores the answers that a user has entered into that form. The form in progress stores the id of the form as a foreign key to form the link. Each form in progress relates to one user because it only stores that single users answers. Each user has

their own unique form in progress database. Each user can have many form in progress objects because they can attempt multiple forms at once.

Generate ID is needed for all entities so that they can be uniquely identified in the system. All entities need to be able to be added. User, form, Question, and QuestionPanels will be able to be edited by the user. FormInProgress objects and questionStat objects will be edited automatically by the system to update the information when the user attempts a form. All entities need to be able to be searched so that they correct ones can be retrieved when required. All should also be able to be saved to file, and read from file, so that when the system is closed and opened again no data is lost.

There needs to be an operation to make a deep copy of a question panel so that it can be saved within a formInProgress without affecting the original questionPanel object. This means that multiple users will be to attempt the same questions without altering each other's progress.

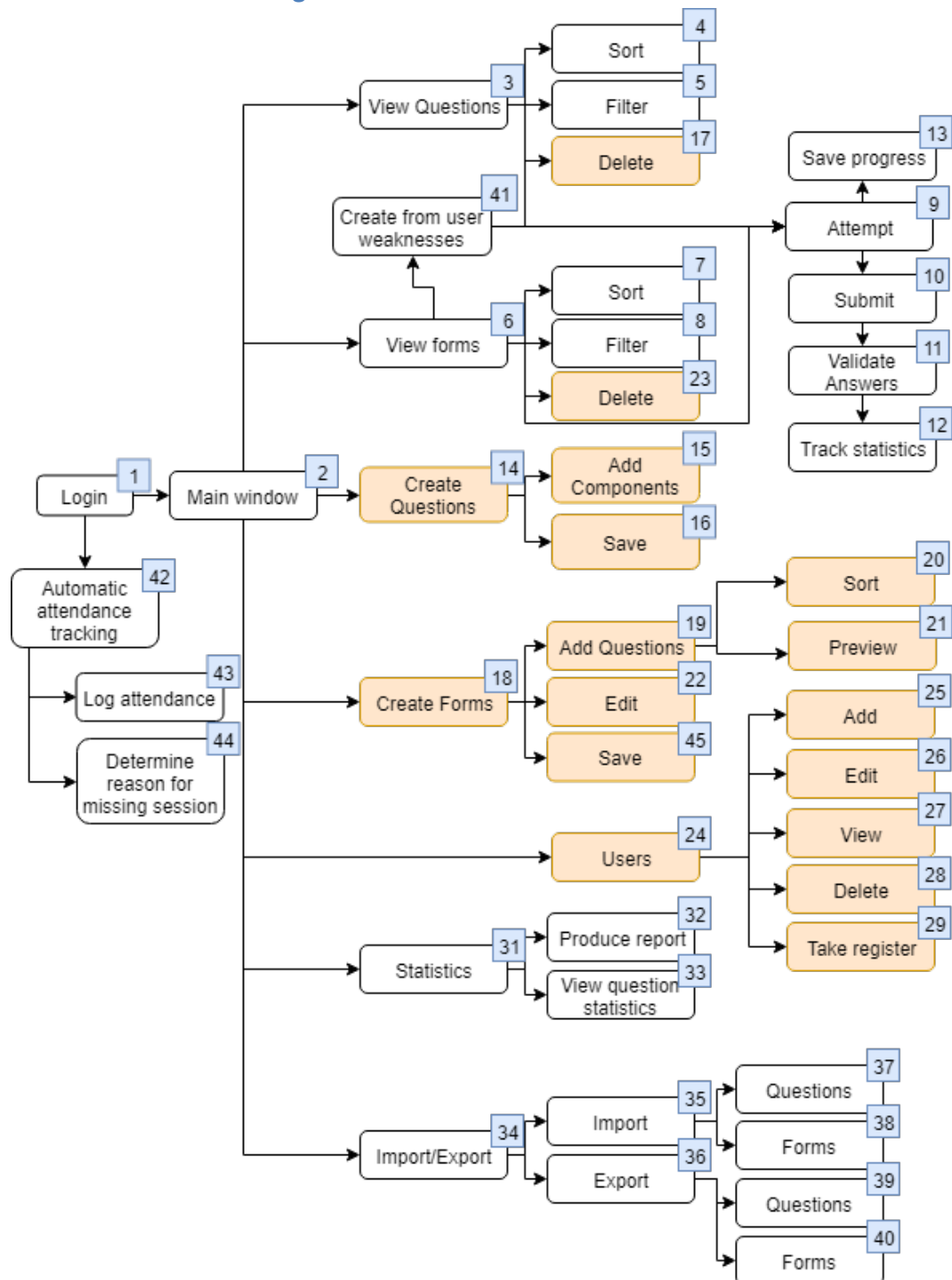
Form needs an operation that retrieves all of the question panels that relate to the questions that are in the form so that they can be displayed.

3.13 Links between Objectives and Sub Problems

Objective # (Investigation)	Objective Description (Investigation)	Sub-Problem # (Breakdown Diagram)
	Questions	
1	Ability to create Questions	14, 15, 16
2	Ability to add labels	15
3	Ability to add text fields	15
4	Ability to specify the type of data that should be entered into the text field	15
5	Ability to add radio buttons	15
6	Ability to add drop-downs	15
7	Ability to add checkboxes	15
8	Ability to add password fields	15
9	Ability to add calendar entry	15
10	Ability to add a document upload field	15
11	Ability to delete Questions	17
12	Ability to sort Questions by difficulty	4
13	Ability to sort Questions by type	4
14	Ability to filter Questions by difficulty	5
15	Ability to filter Questions by type	5
16	Email addresses, dates, and phone numbers should have format checks	11
17	Addresses should be looked up via the google maps API to autofill information	9
18	Passwords should have double entry verification	11
19	All fields should have a presence check	11
20	Document upload fields should only allow certain types of files to be uploaded	11
21	Ability to export a Question to file	39
22	Ability to import a Question from file	37
23	Ability to attempt a question	9
24	Ability to submit a question	10
	Forms	
25	Ability to create forms	18, 19, 45
26	Ability to create forms from user weaknesses	41
27	Ability to sort forms by difficulty	7
28	Ability to filter forms by difficulty	8
29	Ability to filter forms by types of questions tested	8
30	Ability to edit forms	22
31	Ability to delete forms	23
32	Ability to export forms to file	40
33	Ability to import forms from file	38
34	Ability to attempt a form	9
35	Ability to submit a form	10
	User	
36	The time taken to complete each question should be stored	12
37	The number of times that they've failed the validation check for a question should be stored	12
38	Ability to add users	25
39	Ability to edit users	26
40	Ability to delete users	28
41	Ability to search through users	27
42	Ability to produce a report on the user's weak areas/progress	32
43	How much of each form a user has attempted should be tracked.	13
44	First name, last name, Date of birth and phone number should be stored for each user	25
45	The attendance for each user should be automatically tracked	42, 43

46	The system should determine the reason for a user missing a session	44
Interface		
47	Should be simple	-
48	Login page which allows users or admins to login	1
49	There should be a main menu panel	2
50	There should be a panel which will display the forms to the user and allow them to search through them	6
51	Allow user to select a form to be attempted	6
52	Allow form to be submitted	10
53	There should be a panel which will display the questions to the user and allow them to search through them	3
54	Allow user to select a question to be attempted	3
55	Allow question to be submitted	10
56	Questions that have failed their validation check should be highlighted	11
57	The user should be able to create forms by selecting questions from a list	19
58	There should be a statistics screen which shows the user's progress	31
59	There should be a context sensitive help button available in each window and panel of the application	2, 3, 6, 14, 18, 24, 31, 34
60	The admin user should have a different main menu	2
61	The admin user should be able to view information about each user	27
62	The admin user should be able to edit information about each user	26
63	The admin user should be able to take a register	29

3.14 Sub-Problem Diagram



Orange indicates admin only functions.

3.2 Input and Output Designs

3.21 Identification of Data

Objective # (Investigation)	Feature/ Process	Input Required	Output Created	Consideration of Output
48	Login	Username Password	N/A	N/A
49	Open other screens	Buttons for each screen of the system	N/A	N/A
53	Display Questions	List of questions Data from each question: Difficulty Type	A table displaying this information to the user	The type and difficulty of each question is required so that the user can identify the questions in the system.
12	Sort questions by difficulty	Button to initiate sort	A list of questions in order of difficulty	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the sort.
13	Sort questions by type	Button to initiate sort	A list of questions in order of type	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the sort.
14	Filter questions by difficulty	Number (1-10) of difficulty to filter by	A list of questions that only contains the ones of the required difficulty.	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the filter.
15	Filter questions by type	List of types to sort by	A list of questions that only contains the ones of the required type.	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the filter.
11	Delete questions	ID of the question to delete	Updated question list with question to delete removed.	The new question list replaces the old one. This wipes the question to be deleted from the system.
23	Attempt question	ID of the question to attempt	Form with just the question in it	This form is required so that it can be passed to the form attmptter for the user to attempt.
50	Display forms	List of Forms Data from each form:	A table displaying this information to the user	All these attributes from each form are required so that the user can identify the forms in the system and see the data stored about them.

		Description Main skills tested Difficulty Percentage complete		
27	Sort forms by difficulty	Button to initiate sort	A list of forms in order of difficulty	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the sort.
28	Filter forms by difficulty	Number (1-10) of difficulty to filter by	A list of forms that only contains the ones of the required difficulty.	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the filter.
29	Filter forms by main skills tested	The types of questions that the form should contain.	A list of forms that only contains the ones of the required difficulty.	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the filter.
26	Attempt form based on user weaknesses	The user's statistics for each question	A form containing the questions that the user struggles with the most.	The form output is required so that it can be passed to the form attempter so that the user can attempt the form.
34	Attempt Form	ID of the form to attempt List of questions in form (obtained from form database)	A form attempter window containing the form.	The form attempter window is required so that the user can attempt the form.
31	Delete form	ID of the form to delete	Updated form list with form to delete removed.	The new form list replaces the old one. This wipes the form to be deleted from the system.
34	Attempt form window-attempt form	List of questions in form	A window containing all of the question panels of the questions in the form.	The question panels need to be displayed to the user so that they can attempt the question and fill them in.
1	Create questions - question preview	List of components in question	The components are graphically	The components are required so that they can be added to the question preview panel and allow the user to see exactly what the question will look

			presented to the user.	like.
1	Create questions - add component	Component to add	Updated list of components in question	The updated list of components is stored and is required for the question to correctly be displayed because it contains all of the graphical elements of the question. The list is also used to update the question preview.
1	Create questions Save/add question	Type, difficulty, List of components	Question object (written to file)	The question object's attributes have the values dictated by the input. The object is stored in the questions database and can be written to file and added to forms. Using an object to store the data keeps it well organized as each object is a single question.
2	Add label	Label text	Label object	The label object stores the text that the label has and tells the form attmptter how to display it. This is important so that the label is displayed correctly.
3	Add text fields	Type of validation required	Textfield object	The textfield object stores the validation that the textfield has and tells the form attmptter how to display it. This is important so that the textfield is displayed correctly.
5	Add radio buttons	Options	Radiobuttons object	The radio buttons object stores the options that the radio buttons allow to be chosen and tells the form attmptter how to display it. This is important so that the radio buttons are displayed correctly.
6	Add drop-down	Options	drop-down object	The dropdown object stores the options that the dropdown allows to be chosen and tells the form attmptter how to display it. This is important so that the dropdown is displayed correctly.
7	Add check boxes	Options	checkboxes object	The checkboxes object stores the options that the checkboxes allow to be chosen and tells the form attmptter how to display it. This is important so that the checkboxes are displayed correctly.
10	Add file chooser	Types of files to be accepted	filechooser object	The filechooser object stores the validation that the filechooser has and tells the form attmptter how to display it. This is important so that the filechooser is displayed correctly.
57	Create forms - question list	List of questions Data from each question: Difficulty	A table displaying this information to the user	The type and difficulty of each question is required so that the user can identify the questions in the system.

		Type		
25	Create forms - question preview	ID of Question to preview	Graphical preview of the question	The components that form the question panel are displayed to the user so they can see what the question looks like before adding to the form.
57	Create forms - Sort questions by difficulty	Button to initiate sort	A list of questions in order of difficulty	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the sort.
57	Create forms - Sort questions by type	Button to initiate sort	A list of questions in order of type	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the sort.
57	Create forms - add question to form	ID of the question to add to form	An updated list of questions in the form with the question added	The updated list of questions is stored and is required for the form to correctly be displayed because it contains all of the graphical elements of the form. The list is also used to update the form preview.
25	Create forms - form preview	List of questions in the form	The questions are graphically presented to the user.	The questions are required so that they can be added to the form preview panel and allow the user to see exactly what the form will look like.
25	Save forms	Description Difficulty List of questions in form	Form object saved to file	The form object's attributes have the values dictated by the input. The object is stored in the forms database and can be written to file. Using an object to store the data keeps it well organized as each object is a single form.
61	View users	List of users Data from each user: Username Password First name Last name Date of birth Phone Number Sessions attended	A table displaying this information to the user.	The table displays all of the information stored about each question to the user.
41	Search users	First name	List of users with that first name	This list is used as the data to be displayed in the table and the table is updated so that the user can see the

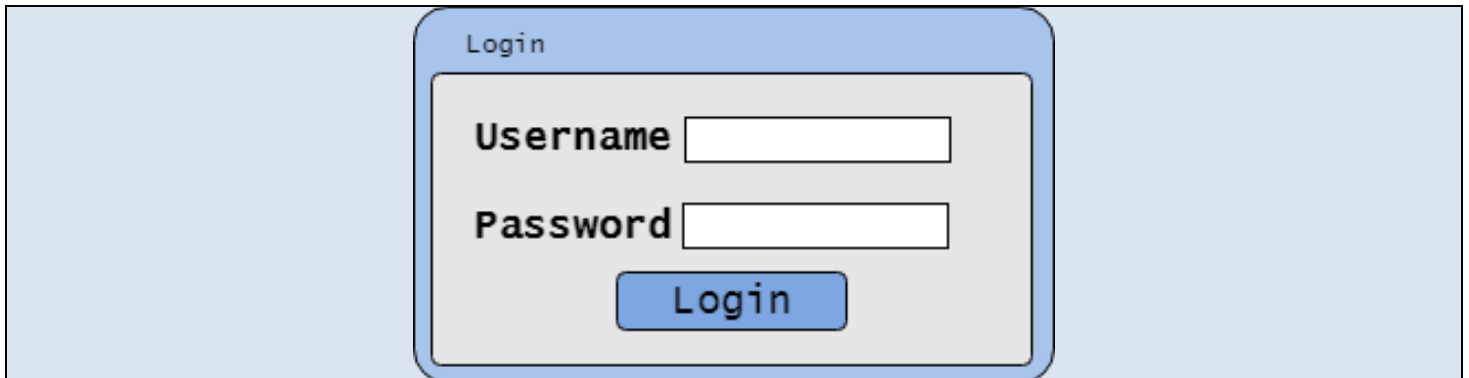
				result of the search.
40	Delete user	User ID	Updated user list with user to delete removed.	The new user list replaces the old one. This wipes the user to be deleted from the system.
38	Add user	Username Password First name Last name Date of birth Phone number	User object saved to file	The user object's attributes have the values dictated by the input. The object is stored in the user database and can be written to file. Using an object to store the data keeps it well organized as each object is a single user.
39	Edit user	Username Password First name Last name Date of birth Phone number Sessions attended	Updated user object	The updated user object replaces the old one in the database.
63	Register - Log attendance	Lists of users	User objects have updated attendance	The updated user objects store the attendance data.
42	Statistics produce - report	QuestionStatList (stores the statistics of all the users attempts of the question) Data about each question: Average time to complete Number of attempts Number of	Report that can be printed. A table displaying all of the statistics broken down by question type. First name Last name Data about each type:	This report makes it easy for the admin/volunteers to see how each user is progressing and how good they are at completing different types of questions. The first and last name of the user are added to the top so that the user which it belongs to can be identified when the report has been printed.

		times failed validation	Number of times failed validation Average number of attempts needed to correct an error Average time to complete	
58	Statistics - view question	The ID of the question	Times failed validation Most recent number attempts taken to correct an error Most recent times taken to complete question Average time taken to complete	This data is used to produce graphs and charts that are used to visually represent the user's progress. The most recent number of attempts taken to correct an error are used to plot a line chart to show the user's progress over time. The most recent times taken to complete are plotted in the same way.
21	Import/Export - question list	List of questions Data from each question: Difficulty Type	A table displaying this information to the user	The type and difficulty of each question is required so that the user can identify the questions in the system.
21	Import/Export - question preview	ID of Question to preview	Graphical preview of the question	The components that form the question panel are displayed to the user so they can see what the question looks like before exporting it.
21	Import/Export - Question type sort	Button to initiate sort	A list of questions in order of type	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the sort.
21	Import/Export - Question difficulty sort	Button to initiate sort	A list of questions in order of difficulty	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the sort.

22	Import/Export - Import Question	Question saved to file	N/A	N/A
21	Import/Export - Export Question	Question to export. Location to export the question.	Exported Question	The exported question can be imported again by a different instance of the system.
32	Import/Export - Form list	List of Forms Data from each form: Description Main skills tested Difficulty Percentage complete	A table displaying this information to the user	All these attributes from each form are required so that the user can identify the forms in the system and see the data stored about them.
32	Import/Export - sort forms by difficulty	Button to initiate sort	A list of forms in order of difficulty	This list is used as the data to be displayed in the table and the table is updated so that the user can see the result of the sort.
32	Import/Export - preview form	List of questions in the form	The questions are graphically presented to the user.	The questions are required so that they can be added to the form preview panel and allow the user to check that they are exporting the correct form because they are able to see what the form looks like.
32	Import/Export - Export form	Form object. Location to export the question.	Exported Form	The exported Form can be imported again by a different instance of the system.
33	Import/Export - Import form	Form saved to file	N/A	N/A
46	Determine reason for missing a lesson	Reason	N/A	N/A

3.22 Designs of Input & Output

The font used throughout these screens is 'Lucida Console' because it's easy to read which is important as the users of the system are elderly and may have sight problems. For this reason, it's also important that the font is large.



THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

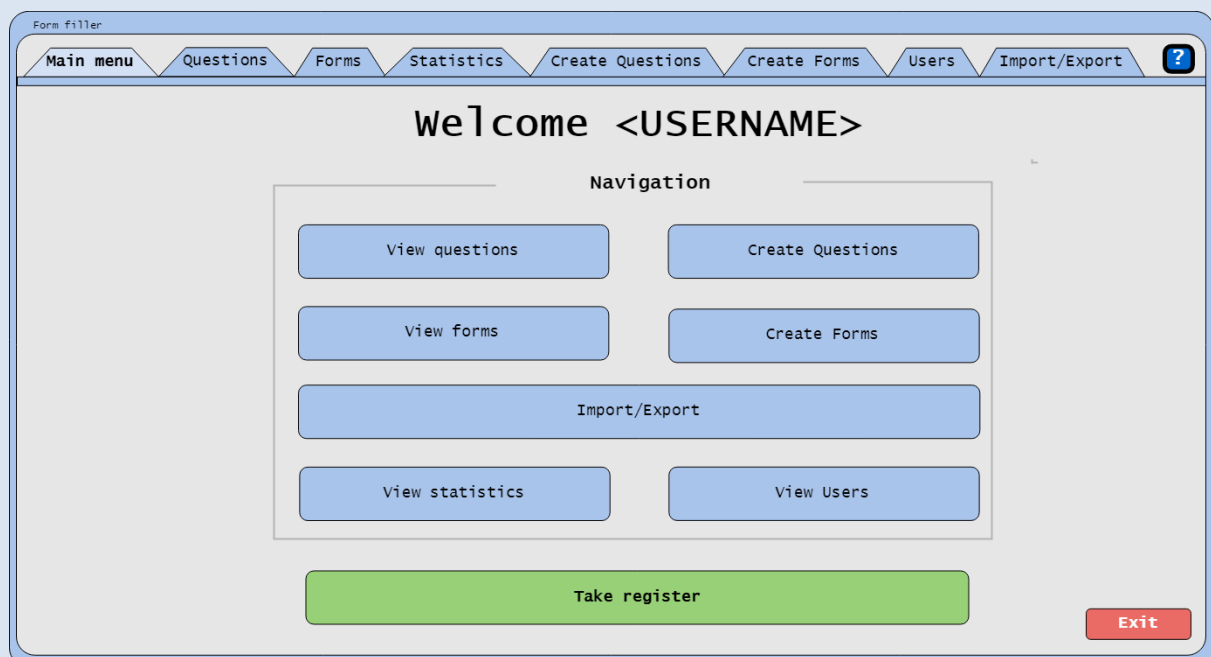
Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Login window	This allows a user to login to the system.	JLabel - "Username" JLabel - "Password" These labels indicate the purpose of their respective fields JTextField - Username field JPasswordField - Password field These fields allow the user to enter their username and password JButton - "Login" - When pressed the username and password are validated and the user is logged in	The username and password labels are size 18 and bold this is so they stand out and immediately draw the user's attention. The login button text is also size 18 but isn't bold. This is so that it doesn't blend in with the username and password labels and remains distinct. It also reduces the clutter of the window.	48

User view



The User view interface features a top navigation bar with tabs: Main menu, Questions, Forms, Statistics, Create Questions, Create Forms, Users, and Import/Export. The main content area displays a welcome message 'Welcome <USERNAME>'. On the left, a 'Navigation' sidebar contains buttons for 'View questions', 'View forms', 'View statistics', and 'Import/Export'. In the center, a 'Continue?' section shows 'Last form attempted: <FORMID> <FORMDESCRIPTION>' with a green 'Resume form' button. On the right, an 'Attendance' section shows a circular progress indicator at 20%. A red 'Exit' button is located at the bottom right.

Admin view



The Admin view interface features a top navigation bar with tabs: Main menu, Questions, Forms, Statistics, Create Questions, Create Forms, Users, and Import/Export. The main content area displays a welcome message 'Welcome <USERNAME>'. In the center, a 'Navigation' section contains buttons for 'View questions', 'Create Questions', 'View forms', 'Create Forms', 'Import/Export', 'View statistics', and 'View Users'. A large green 'Take register' button is positioned below the navigation section. A red 'Exit' button is located at the bottom right.

THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Main screen	This allows a user to login to the system.	User + Admin JLabel - "Welcome <username>" - This label clearly indicates to the user which user is currently logged in. Border - "Navigation" - This border is used	The "Welcome Username" text is bold and size 46 so that it stands out and draws the users attention. The text of all the borders is size 18 and bold this is so	49, 59, 60

		to indicate the buttons that are used to navigate the system to the user. JButtons - All the buttons in the navigation section take the user/admin to the screen that their text indicates. JButton - "Exit" - This button allows the user to close the system. (This has the same function as the close button of the window) User Border - "Continue?" - This border is used to indicate the components that are used to continue the most recent form that they attempted to the user. JLabel - "Last form attempted: <FORMID> <FORMDESCRIPTION>" - This label indicates to the user what the last form that they attempted was by showing them the id and description of the form. JButton - "Resume form" - This button allows the user to resume attempting the form that they were most recently attempting. JBorder - "Attendance" - This border is used to indicate that the pie chart shows the user's attendance. JFreeChart - Attendance pie chart - this indicates to the user what percentage of the previous sessions they have been present at. Admin JButton - "Take register" - This button immediately opens a register for the admin to take. This is to save the admin time so that they don't have to open the user screen of the application and open the register from there.	they are easy to read and stand out from the text of the buttons. The text of the navigation buttons is size 18 and not bold. As there is a large amount of buttons, using text that is large but not bold means that it is still easy to read but the buttons do not look cluttered. "Last form attempted: <FORMID>" is size 18 and bold so that it is easy to read and stands out from the form description. The form description text is also size 18 but not bold. The "Resume form", "Take register" and "Exit" button have bold text to indicate to the user that they perform significant actions. The "Resume Form" button is green to show the user that this is the final action that they will take on the screen. It also makes the button more approachable. The text is black because it's easy to read. The "Take register" button is green to show the admin that the button performs the "start" action, as in it is the first action that should be taken each session. It also makes the button more approachable. The text is black because it's easy to read. The "Exit" button is red to make it stand out and to indicate to the user that it performs a destructive action (exiting the program) and so warns them about pressing it. The text of the button is bold and also white in order to provide adequate contrast from the red and make it easy to read.	
--	--	--	---	--

User view

Form filler

Main menu Questions Forms Statistics Import/Export ?

Type	Difficulty
Email	4
Birthday	6
Address	2
Dropdown	4
Photo upload	7
Name	2
Address	4
Email	2

Sort and Filter

Sorts

Difficulty sort

Type sort

Difficulty filter

1 2 3 4 5 6 7 8 9 10

Apply difficulty filter

Type filter

☒ Email ☐ Birthday ☐ Address

☐ Dropdown ☐ Photo upload ☐ Name

Apply type filter

Reset sorts and filters

Attempt question

Admin view

Form filler

Main menu Questions Forms Statistics Create Questions Create Forms Users Import/Export ?

Type	Difficulty
Email	4
Birthday	6
Address	2
Dropdown	4
Photo upload	7
Name	2
Address	4
Email	2

Sort and Filter

Sorts

Difficulty sort

Type sort

Difficulty filter

1 2 3 4 5 6 7 8 9 10

Apply difficulty filter

Type filter

☒ Email ☐ Birthday ☐ Address

☐ Dropdown ☐ Photo upload ☐ Name

Apply type filter

Reset sorts and filters

Delete question

Attempt question

THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
View Question	This allows a user to browse (sort and	JTable - This is used to display the questions along with their type and	The border labels have bold text that is slightly	11, 12, 13, 14, 15, 53,

s screen	filter) the questions in the system and select one to attempt.	difficulty. It allows multiple data items that are related to multiple entities to be easily displayed and grouped by entity. Each row is a different entity (question) and the columns is the data belonging to it. The user can also select a row and this allows them to select a question to attempt JScrollPane - This allows the JTable to be scrolled when it's full and the contents can't all fit on the screen at once. Border - "Sort and filter" - Border marks the region of the screen that contains the sorts and filters. JLabel - "Sorts" - This tells the user which buttons are used to sort the question list JButton - "Difficulty Sort" and "Type Sort" - These buttons allow the user to sort the table when they are pressed. JLabel - "Difficulty filter" - this makes the user aware of which components are used to perform a difficulty filter. JSlider - This allows the user to easily enter a number between 1 and 10 to filter by. It also means that the user can't enter an invalid number. This makes the interface less confusing to the user as they can't enter invalid options. JButton - "Apply difficulty filter" - This button allows the user to apply the difficulty filter. JLabel - "Type filter" - This makes the user aware of which components are used to perform a type filter. JRadioButton - There is a radio button for each type and this allows the user to filter the table down to just 1 type. A radiobutton is used because it only allows the user to select a single type at a time. This is used because each question only has 1 type. JButton - "Reset sorts and filters" - This button is used to easily reset all of the sorts and the filters that the user has applied. JButton - "Attempt question" - This allows the user to open a question that they have selected in the table to be attempted.	larger than the other text (size 22). This is to make them stand out from the other text. All other text is size 18, apart from the labels for each radio button which are size 16. The radio button labels are smaller than the other text in order to reduce clutter as there will be a large number of radio buttons. The headings for the table are bold in order to differentiate them from the data in the table. The headings are size 16 while the data in the table is size 18. The data is size 18 so that it can easily be read. The headings are smaller because they are less important and the smaller size means that there is more room for the data. The "Reset sorts and filters" and "Attempt question" button have bold text to indicate to the user that they perform significant actions. The "Reset sorts and filters" button is red to make it stand out and to indicate to the user that it performs a destructive action (resetting the sorts) and so warns them about pressing it. The text of the button is white in order to provide adequate contrast from the red and make it easy to read. The "Attempt question" button is green to show the user that this is the final action that they will take on the screen. It makes them aware that by pressing it they're marking the screen as 'done' or 'complete'. It also makes the button more approachable. The text is	54, 59
----------	--	--	---	--------

		Button - “Delete question” - This allows the admin to delete the question that is selected in the table	black because it’s easy to read. The “Delete question” button is also red to indicate that it performs a destructive action and the text is white so that it’s easy to read. A darker red is used so that it can be differentiated from the “Reset sorts and filters” button at a glance. All other buttons are blue in order to match with the theme of the application and not distract the user.	
--	--	---	--	--

User view

Form filler

Main menuQuestionsFormsStatisticsImport/Export

Description	Main skills tested	Difficulty	Percentage complete
Filling in basic personal info	Name, Dropdown, Birthday, Radiobuttons, Email Address	4	50%
Blue badge practice	Name, Dropdown, Address, Radiobuttons, Email Address, Document upload, Date entry	6	25%

Sort and Filter

Sorts

Difficulty sort

Difficulty filter

12345678910

Apply difficulty filter

Main skills tested filter

☒ Email☐ Birthday☐ Address

☐ Dropdown☒ Document upload☐ Name

Apply main skills tested filter

Reset sorts and filters

Attempt Form based on weaknesses

Attempt Form

Admin view

THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
View Forms screen	This allows a user to browse (sort and filter) the forms in the system and select one to attempt.	JTable - This is used to display the forms along with their description, main skills tested, difficulty, and percentage complete. It allows multiple data items that are related to multiple entities to be easily displayed and grouped by entity. Each row is a different entity (form) and the columns is the data belonging to it. The user can also select a row and this allows them to select a form to attempt JScrollPane - This allows the JTable to be scrolled when it's full and the contents can't all fit on the screen at once. Border - "Sort and filter" - This border marks the region of the screen that the contains the sorts and filters. JLabel - "Sorts" - This tells the user which buttons are used to sort the form list JButton - "Difficulty Sort" - This button allows the user to sort the table when they are pressed. JLabel - "Difficulty filter" - this makes the user aware of which components are used to perform a difficulty filter. JSlider - This allows the user to easily enter a number between 1 and 10 to	The border labels have bold text that is slightly larger than the other text (size 22). This is to make them stand out from the other text. All other text is size 18, apart from the labels for each checkbox which are size 16. The checkbox labels are smaller than the other text in order to reduce clutter as there will be a large number of checkboxes. The headings for the table are bold in order to differentiate them from the data in the table. The "Reset sorts and filters", "Attempt Form based on weaknesses", and "Attempt Form" button have bold text to indicate to the user that they perform significant actions. The "Reset sorts and filters" button is red to make it stand out and to indicate to the user that it performs a	26, 27, 28, 29, 31, 50

		filter by. It also means that the user can't enter an invalid number. This makes the interface less confusing to the user as they can't enter invalid options. JButton - "Apply difficulty filter" - This button allows the user to apply the difficulty filter. JLabel - "Main skills tested filter" - This makes the user aware of which components are used to perform a "Main skills tested" filter. JCheckBox - There is a checkbox for each skill (type of question in the form) and this allows the user to filter the table down to the forms that contain the types that they have selected to filter by. Checkboxes are used because each form has multiple skills that it tests and checkboxes allow multiple skills to be selected. JButton - "Reset sorts and filters" - This button is used to easily reset all of the sorts and the filters that the user has applied. JButton - "Attempt Form based on weaknesses" - This triggers the creation of a form from the user's weaknesses and opens it for them to attempt. A button is used because it provides as easy, one click, way of invoking this action. JButton - "Attempt form" - This allows the user to open the form that they have selected in the table to be attempted. JButton - "Delete form" - This allows the admin to delete the form that is selected in the table	destructive action (resetting the sorts) and so warns them about pressing it. The text of the button is white in order to provide adequate contrast from the red and make it easy to read. The "Attempt Form based on weaknesses" button is purple to clearly differentiate it from the green "Attempt Form" button. Purple was chosen because it makes the button stand out and doesn't clash with the green. The text is black because it's easy to read. The "Attempt Form" button is green to show the user that this is the final action that they will take on the screen. It makes them aware that by pressing it they're marking the screen as 'done' or 'complete'. It also makes the button more approachable. The text is black because it's easy to read. The "Delete form" button is also red to indicate that it performs a destructive action and the text is white so that it's easy to read. A darker red is used so that it can be differentiated from the "Reset sorts and filters" button at a glance. All other buttons are blue in order to match with the theme of the application and not distract the user.	
--	--	---	--	--

THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Create questions screen	This allows the admin to create new questions and add them to the system	Border - "Question information" - This border is used to indicate to the user which components are used to fill in question information. JLabel - "Question Type" and "Question difficulty" - These labels are used to indicate the purpose of their respective comboboxes. JComboBox - Question type and difficulty - These comboboxes are used to allow the user to enter the questions type and difficulty. ComboBoxes are used because, as there are only fixed question types and difficulties, the user doesn't need to enter custom data and so by using a combobox the user's input doesn't need to be validated. JButton - "New type" - This button allows the user to add a new question type to the combobox if the type that they want is available. Border - "Add components" - This border indicates to the user which components are used to fill in question information. JButton - Add components buttons - There is a button for each type of component the user can add to the question. These	The border labels have bold text that is slightly larger than the other text (size 22). This is to make them stand out from the other text. The "Question type" and "Question difficulty" labels are the same as the headings labels to make them easy to read. The text in the buttons is not bold and is size 18. This is because this size is still easy to read and reduces clutter. There wouldn't be enough room for the text if it was larger and bold. The "Reset Question" button is red to make it stand out and to indicate to the user that it performs a destructive action and so warns them about pressing it. The text of the button is white in order to provide adequate contrast from the red and make it easy to read. The "Save Question" button is	1, 2, 3, 4, 5, 6, 7, 8, 9, 10

		are used so that the user can easily add new components. Unlike dragging and dropping, the user doesn't need precision or accuracy using a mouse or the time to drag it across the screen. This means that the user can create questions much more accurately and quickly. JButton - "Reset Question" - This allows the user to reset the question and remove all added components with a single click JButton - "Save Question" - This allows the user to save the created question with a single click Border - "Question preview" - This border is used to indicate the region of the screen that is the question preview to the user JPanel - Question preview - This JPanel is used because the components of the question can be added to it and allows the question to be visually previewed.	green to show the user that this is the final action that they will take on the screen. It makes them aware that by pressing it they're marking the screen as 'done' or 'complete'. It also makes the button more approachable. The text is black because it's easy to read.	
--	--	---	---	--

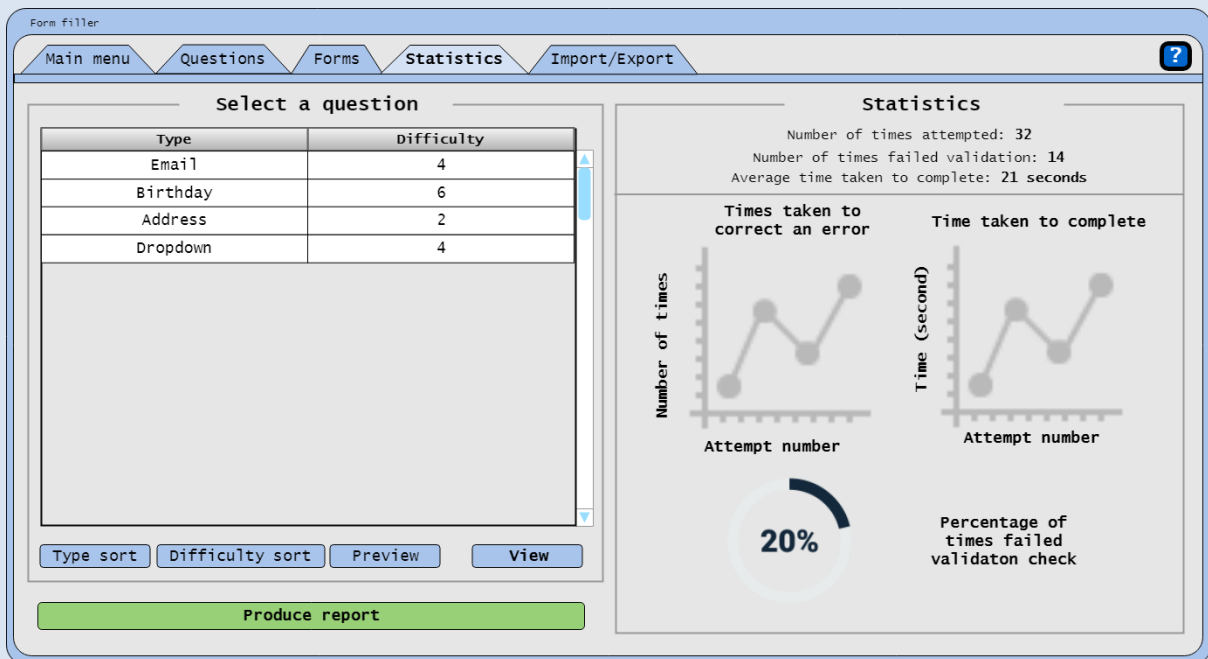
THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Create forms screen	This allows the admin to create new forms and edit existing	Border - "Edit forms" - This indicates the components which are used to load a form for editing. JLabel - "Select a form to edit" - This	The border labels have bold text that is slightly larger than the other text (size 22). This is to make them stand out from	48

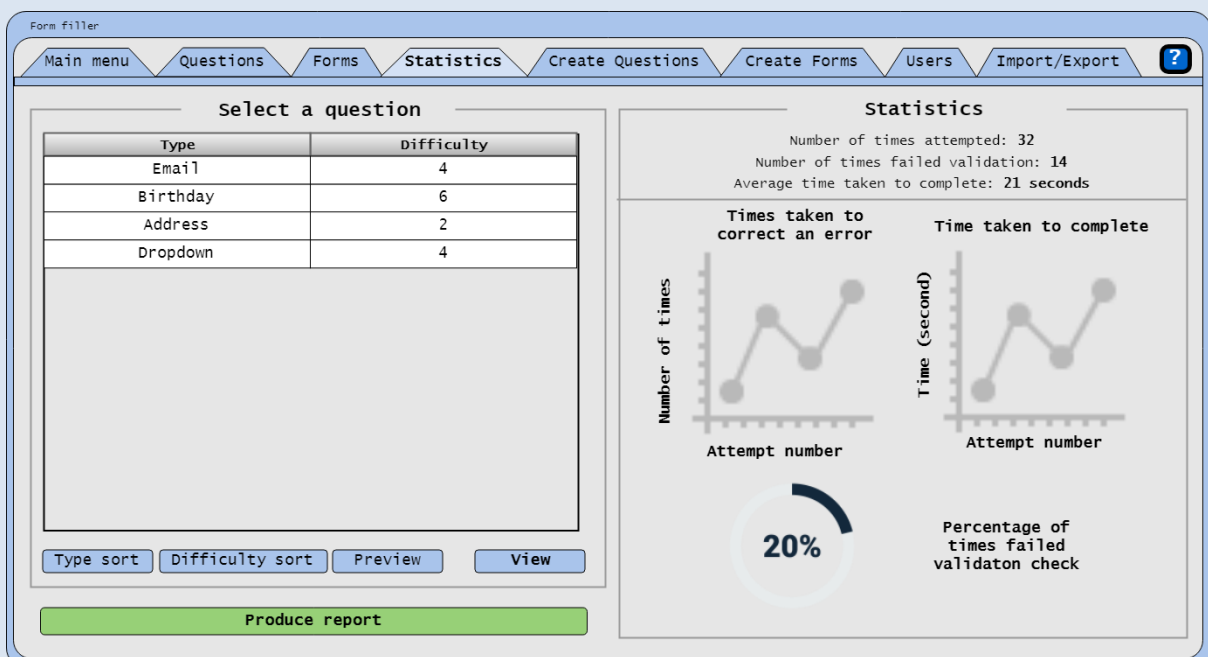
	forms.	makes the purpose of the combobox to its right clear JComboBox - This combobox allows the user to select a form from the system to load and edit JButton - "Edit" - the user can press this button to load the selected form for editing. Border - "Form information" - This indicates the components which are used to enter information about the form. JLabel - "Form difficulty" - This indicates the purpose of the combobox to its right to the user JComboBox - This lets the user enter the forms difficulty. This is the most suitable component as the form difficulty can only be an integer between 1 and 10 and so using a combobox means that the user's input doesn't have to be validated. JLabel - "Form description" - This indicates the purpose of the JTextArea to its right to the user. JTextArea - This allows the user to enter a multiline description of the form. A JTextArea is used rather than a JTextField because multiple lines of entry are required and a JTextField only supports one line. Border - "Select questions" - This indicates the components that are used to add questions to the form to the user. JTable - This is used to display the questions along with their type and difficulty. It allows multiple data items that are related to multiple entities to be easily displayed and grouped by entity. Each row is a different entity (question) and the columns is the data belonging to it. The user can also select a row and this allows them to select a question JScrollPane - This allows the JTable to be scrolled when it's full and the contents can't all fit on the screen at once. JButton - "Difficulty Sort" and "Type Sort" - These buttons allow the user to sort the table when they are pressed. JButton - "Preview" - This button opens the selected question to preview when pressed. JButton - "Add" - This adds the selected question to the form. JButton - "Reset form" - This allows the user to reset the form and remove all added questions with a single click.	the other text. The headings for the table are bold in order to differentiate them from the data in the table. The headings are size 16 while the data in the table is size 18. The data is size 18 so that it can easily be read. The headings are smaller because they are less important and the smaller size means that there is more room for the data. The text on the buttons below the table is size 18. This is because it's easy to read. The text of the add button is bold to make it stand out from the other buttons. The "Reset Form" button is red to make it stand out and to indicate to the user that it performs a destructive action and so warns them about pressing it. The text of the button is white in order to provide adequate contrast from the red and make it easy to read. The "Save Form" button is green to show the user that this is the final action that they will take on the screen. It makes them aware that by pressing it they're marking the screen as 'done' or 'complete'. It also makes the button more approachable. The text is black because it's easy to read.	
--	--------	---	---	--

		JButton - "Save form" - This allows the user to save the form to the database Border - "Form preview" - This indicates to the user the region of the screen which is the form preview JPanel - Form preview - This JPanel is used because the questions of the form can be added to it and allows the form to be visually previewed. JButton - Remove question button (red cross button), and move question up and down buttons (arrow buttons) - JButtons are used here because they allow icons to be clicked to perform actions. This makes the purpose of the buttons clear. There wouldn't be enough room to write text in the buttons as they are so small.		
--	--	---	--	--

User view



Admin view



THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Statistics screen	This allows the user to view statistics about their past attempts completing a question. It also	Border - "Select question" - This indicates the components that are used to select a question to the user. JTable - This is used to display the questions along with their type and difficulty. It allows multiple data items that are related to multiple entities to be	The border labels have bold text that is slightly larger than the other text (size 22). This is to make them stand out from the other text. The headings for the table	42, 58

	allows them to produce a progress report.	easily displayed and grouped by entity. Each row is a different entity (question) and the columns is the data belonging to it. The user can also select a row and this allows them to select a question JScrollPane - This allows the JTable to be scrolled when it's full and the contents can't all fit on the screen at once. JButton - "Difficulty Sort" and "Type Sort" - These buttons allow the user to sort the table when they are pressed. JButton - "Preview" - This button opens the selected question to preview when pressed. JButton - "View" - This updates the statistics section with the information from the selected question. JButton - "Produce report" - This allows the user to produce a progress report. Border - "Statistics" - This indicates to the user the components that show the statistics for the selected question JLabel - "Number of times attempted", "Number of times failed validation" and "Average time to complete" - used to show the user quantitative data about their past attempts completing the question. JSeparator - Used to provide visual separation between the charts and the JLabels. JFreeChart - "Times taken to correct an error" and "Time taken to complete" - These allow line charts to be plotted to show the user's progress over time JFreeChart - "Percentage of times failed validation check" - This allows the percentage of times that the user fails the validation check to be visually represented.	are bold in order to differentiate them from the data in the table. The headings are size 16 while the data in the table is size 18. The data is size 18 so that it can easily be read. The headings are smaller because they are less important and the smaller size means that there is more room for the data. The text on the buttons below the table is size 18. This is because it's easy to read. The text of the "view" button is bold to make it stand out from the other buttons. The "Produce report" button is bold to show that it performs a significant action. It is also green to show that it's the last action that the user will take on the screen. The JLabels that are at the top of the statistics section use font size 16 as it's a good balance between readability and minimising clutter. The actual data of each label is in bold to show that it's the important part of each label and to draw the user's attention to it. All other text in the statistics section (the charting headings and labels) is size 18 and bold to make it easy to read. There is no need for a combination of bold and regular text here because the charts draw enough attention by themselves.	
--	---	--	---	--

Form filler

Main menu Questions Forms Statistics Create Questions Create Forms Users Import/Export ?

Username	Password	First Name	Last name	Date of birth	Phone number	Sessions attended
bob123	bob	bob	bobbins	01/04/03	12345678	29/7/18, 14/08/18, 01/08/18
fmaccourt1	p00lwgp6	Forster	MacCourt	29/04/1987	2478200469	20/08/18, 19/08/18, 14/08/18, 01/08/18
lruller2	password123	Leela	Ruller	19/03/1967	2386249416	29/7/18, 14/08/18, 01/08/18, 27/07/18
alucien3	cheese	Ash	Lucien	26/07/1991	5851009564	19/08/18, 02/08/18, 01/08/18
obarnsley4	octopus	Oralee	Barnsley	16/08/1970	9243490532	29/7/18, 20/08/18, 05/08/18, 01/08/18, 27/07/18
ksiggens5	pencil121	Karyl'n	Siggens	12/04/1960	4059214260	19/08/18, 05/08/18, 01/08/18

Take register First name to search for: Search Edit table Add new user Delete user Save changes

THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Users screen	This allows the admin to view the information stored about each user, and take a register.	JTable - This is used to display the users along with the information stored about them. It allows multiple data items that are related to multiple entities to be easily displayed and grouped by entity. Each row is a different entity (user) and the columns is the data belonging to it. The user can also select a row and this allows them to select a user. JScrollPane - This allows the JTable to be scrolled when it's full and the contents can't all fit on the screen at once. JButton - "Take register" - This allows the user to open the register dialog JLabel - "First name to search for: " - This indicates the purpose of the JTextField to its right to the user. JTextField - This allows the user to enter a first name to search for JButton - "Search" - This allows the user to initiate a search. JButton - "Edit table" - This allows the user to put the table into edit mode and allow each cell's contents to be changed JButton - "Add new user" - This allows the user to add a new user to the system and adds a new blank row to the table JButton - "Delete user" - This allows the user to delete the currently selected user from the system JButton - "Save changes" - This allows the	The headings for the table are bold in order to differentiate them from the data in the table. The headings are size 16 while the data in the table is size 18. The data is size 18 so that it can easily be read. The headings are smaller because they are less important and the smaller size means that there is more room for the data. The text on the buttons below the table is size 18. This is because it's easy to read. The text of the "Take register", "Edit table", and "Save changes" buttons are bold to make them stand out from the other buttons. The "First name to search for:" label is bold to make it stand it out from the grey background and to	38, 39, 40, 41, 44, 61, 62, 63

user to write the changes that they have made to the table to file

make it easier to read. It's also size 18.

User view

The User view interface is titled "Form filler" and includes a navigation bar with "Main menu", "Questions", "Forms", "Statistics", and "Import/Export". It features two main panels: "Questions" and "Forms".

Questions Panel: Contains an "Import" button, a table with columns "Type" and "Difficulty", and buttons for "Type sort", "Difficulty sort", "Preview", and "Export".

Type	Difficulty
Email	4
Birthday	6
Address	2
Dropdown	4

Forms Panel: Contains an "Import" button, a table with columns "Description", "Main skills tested", and "Difficulty", and buttons for "Difficulty sort", "Preview", and "Export".

Description	Main skills tested	Difficulty
Filling in basic personal info	Name, Dropdown, Birthday, Radiobuttons, Email Address	4
Blue badge practice	Name, Dropdown, Address, Radiobuttons, Email Address, Document upload, Date entry	6

Admin view

The Admin view interface is titled "Form filler" and includes a navigation bar with "Main menu", "Questions", "Forms", "Statistics", "Create Questions", "Create Forms", "Users", and "Import/Export". It features two main panels: "Questions" and "Forms".

Questions Panel: Contains an "Import" button, a table with columns "Type" and "Difficulty", and buttons for "Type sort", "Difficulty sort", "Preview", and "Export".

Type	Difficulty
Email	4
Birthday	6
Address	2
Dropdown	4

Forms Panel: Contains an "Import" button, a table with columns "Description", "Main skills tested", and "Difficulty", and buttons for "Difficulty sort", "Preview", and "Export".

Description	Main skills tested	Difficulty
Filling in basic personal info	Name, Dropdown, Birthday, Radiobuttons, Email Address	4
Blue badge practice	Name, Dropdown, Address, Radiobuttons, Email Address, Document upload, Date entry	6

THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

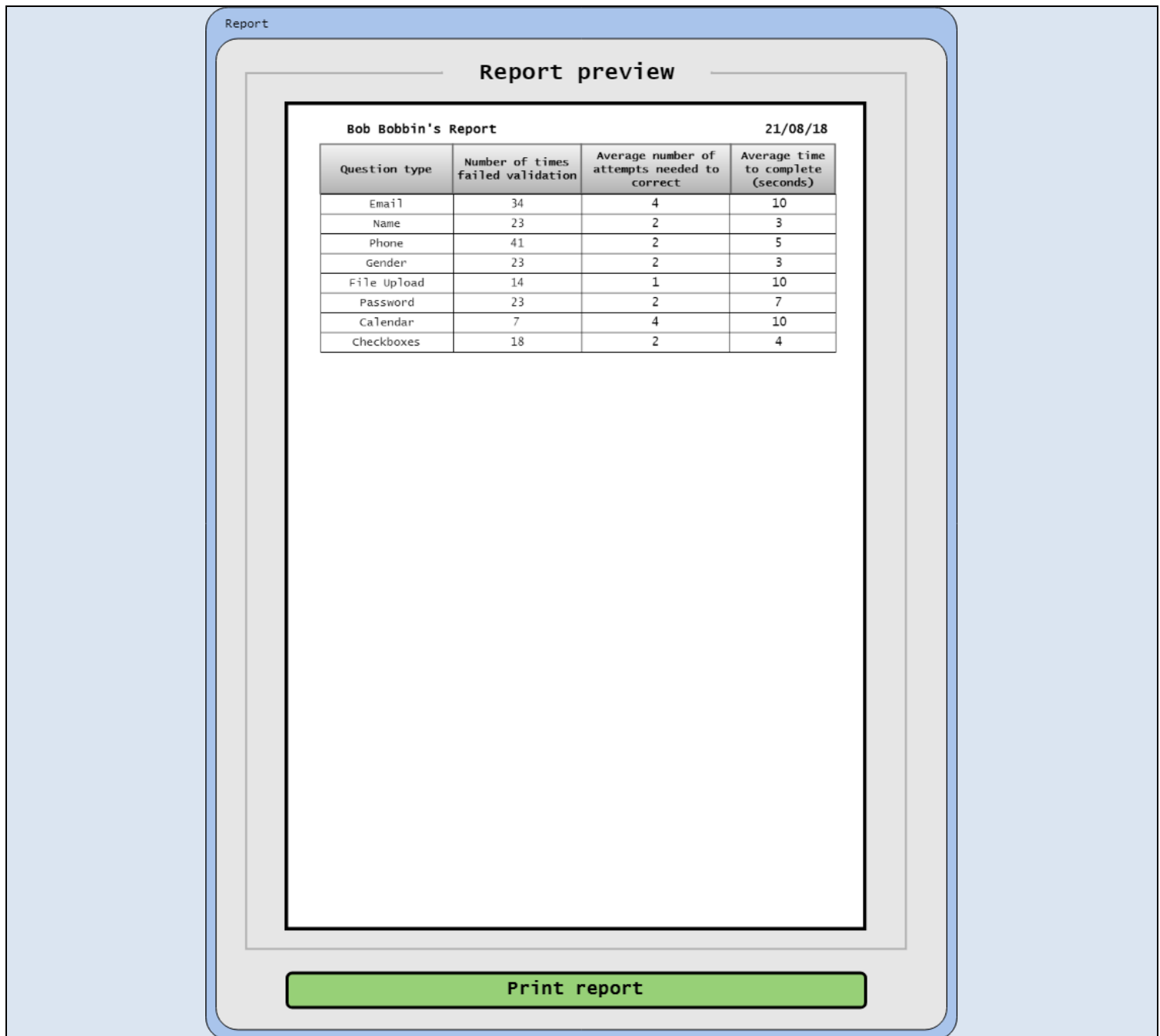
Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Import/Export screen	Allows questions and forms to be imported and exported from	Border - "Questions" - This border indicates to the user which components are used to export and import questions.	The labels that serve as headings have bold text that is slightly larger than the other text (size 22).	21, 22, 32, 33

	the system.	JButton - "Import" - This allows the user to open a JFileChooser to choose a question to be imported. JTable - This is used to display the questions along with their type and difficulty. It allows multiple data items that are related to multiple entities to be easily displayed and grouped by entity. Each row is a different entity (question) and the columns is the data belonging to it. The user can also select a row and this allows them to select a question JScrollPane - This allows the JTable to be scrolled when it's full and the contents can't all fit on the screen at once. JButton - "Difficulty Sort" and "Type Sort" - These buttons allow the user to sort the table when they are pressed. JButton - "Preview" - This button opens the selected question to preview when pressed. JButton - "Export" - This opens a JFileChooser to allow the user to export the selected question. The components are the same for the forms section of the screen.	This is to make them stand out from the other text. The headings for the table are bold in order to differentiate them from the data in the table. The headings are size 16 while the data in the table is size 18. The data is size 18 so that it can easily be read. The headings are smaller because they are less important and the smaller size means that there is more room for the data. The text of the buttons is size 18 because it's easy to read. The "Import" and "Export" buttons use bold text to differentiate them from the other buttons as they are the most important buttons on the screen. The "Import" buttons are green to draw the user's attention to them as they are likely to be the most used buttons on the screen as the user won't be exporting questions or forms very often. The text is white so that it is easy to read on the green button. The "Export" buttons are a slightly darker shade of blue to help differentiate them further from the other buttons.	
--	-------------	--	--	--



THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Register Dialog	This allows the admin to take a register and mark which users are present at a particular session.	JCheckBox - These are used to mark whether a user is absent or not. A checkbox is ideal because it is very easy to use and allows the admin to quickly take a register. A JCheckBox also has a built in label and so allows an easy way to indicate to the user which checkbox is for which user. JPanel - This contains all of the JCheckBoxes and provides a way of arranging and grouping them JScrollPane - This contains the JPanel and provides a way of scrolling it so that more users can fit in the JPanel without having to make the window larger.	The JCheckBox Labels are size 18 so that they are easy to read. The "Save register" button is bold to show that it performs a significant action. It is also green to show that it's the last action that the user will take on the screen. The text is also size 18.	63



THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Print report dialog	This allows the user to print their report. It also provides a print preview so they know what the report is going to look like when printed.	JLabel - "Report preview" - This indicates to the user which area of the dialog contains the report preview. JPanel - This is used to contain the preview of the report JButton - This allows the user to open the java print dialog and print the report.	The border has bold text that is slightly larger than the other text (size 22). This is to make it stand out from the other text. The "Print report" button is bold to show that it performs a significant action. It is also green to show that it's the last action that the user will	42

			take on the screen. The text is also size 18.	
--	--	--	---	--

	Bob Bobbin's Report		21/08/18	
	Question type	Number of times failed validation	Average number of attempts needed to correct	Average time to complete (seconds)
	Email	34	4	10
	Name	23	2	3
	Phone	41	2	5
	Gender	23	2	3
	File Upload	14	1	10
	Password	23	2	7
	Calendar	7	4	10
	Checkboxes	18	2	4

THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Progress report	This is a report that can be printed out detailing stats about the user	JLabel - "<First name> <Last name>'s report" - This label is used to indicate whose report it is. This is so that reports can be easily identified when printed out. JLabel - "<Current date>" - This is to indicate the date of the report this is so that the reports can be easily organised by date when printed out and means that the date of	The JLabels and the headings of the table are bold to make them easy to read and stand out from the data in the table. The JLabels are size 18 so that they are easy to	42

		the report can be determined when printed out. JTable - This is used to display the user's stats, broken down by question type. It allows the data to be clearly displayed and related to different question types as each type has its own row.	read. The headings of the table are size 16 so that they fit in the headings cells of the table. The data in the table is size 16 also but not bold as this is a good compromise between readability and the amount of data that can be fit in the report.	
--	--	---	---	--

Regular page

Attempt form

2 of 4

Name

Gender

☐
☐

Male

☐

Female

Other

Password

Password

Country

 ▼

Address line 1

Phone number

Previous

Next

Last page

Attempt form

4 of 4

Name

Gender ☐ Male ☐ Female
☐ Other

Password

Password

Country ▼

Address line 1

Phone number

[Previous](#) [Submit form](#)

Page with errors

Attempt form 2 of 4

There are problems with the following questions:
Password: Passwords do not match

Please correct them before continuing

Name

Gender ☒ Male ☐ Female
☐ Other

Password

Password

Country

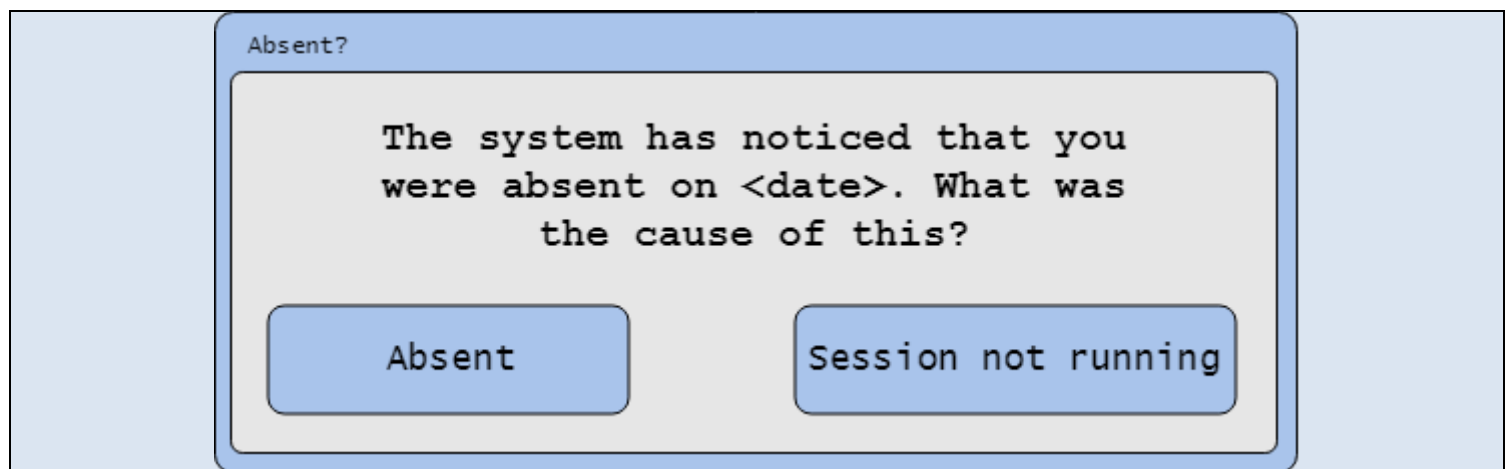
Address line 1

Phone number

THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Attempt form window	This allows a user to attempt a form	JLabel - "<current page> of <total pages>" - This is used to indicate to the user which page of the form the user is attempting. JPanel - This is used to contain all of the questions QuestionPanel - This is used to contain the components for a single question. JButton - "Previous" - This button appears on all pages apart from the first and allows the user to navigate back to the previous page of the form. JButton - "Next" - This buttons appears on all pages apart from the last and allows the user to navigate to the next page in the form.	The page indicator label is size 20 so that it can easily be read. It's regular text rather than bold so that it blends in more and doesn't stand out. The text of the "Previous" and "Next" buttons is not bold and size 18. It's size 18 so that it's easy to read. It's not bold so that it doesn't blend in with the bold labels of the	16, 17, 18, 19, 20, 23, 24, 34, 35, 52, 55, 56

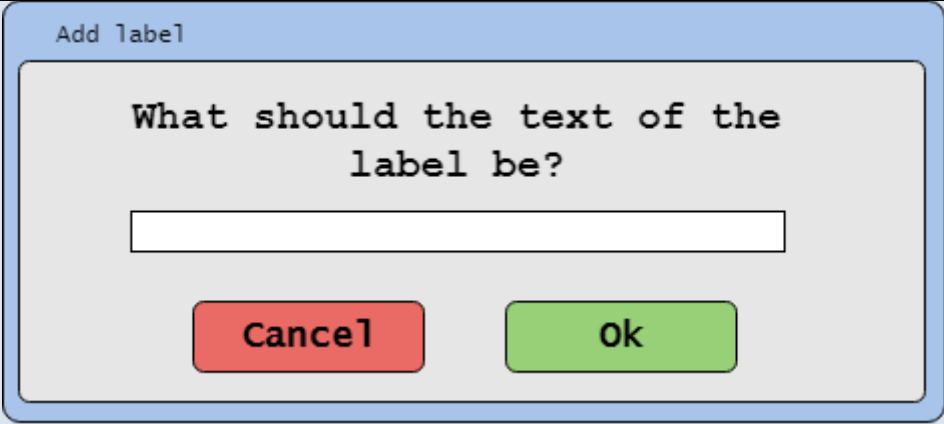
		When the button is pressed the user's answers to the questions on the page are also validated. JButton - "Submit" - This allows the user to submit a form. It triggers the validation of the questions on the page. JLabel - "<Information about the form failed to submit>" - This is used to explain to the user why the questions failed the validation checks. Border - The red borders of the JLabel and questions that the user has answered incorrectly are used to draw the user's attention and indicate to them that they've answered the questions incorrectly.	questions and so is differentiated from the questions. The "Submit form" button is bold to show that it performs a significant action. It is also green to show that it's the last action that the user will take on the screen. The text is also size 18. The text of the JLabel that reports what questions are incorrect is size 20 so that it is easy to read. The text that actually describes what is wrong with the questions is bold so that it stands out and means that the user can immediately pick out the important text from the label.	
--	--	---	---	--



THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Reason for missing a session dialog	This screen opens when the system detects that the user hadn't logged in on a particular date that they should have. This allows the user to specify whether this was because the session wasn't running or because they were absent.	JLabel - This label describes why the dialog has opened to the user. JButton - These buttons allow the user to easily specify whether the session was running or not. This is preferable to a text field because it's much faster for the user to press a button rather than enter text. Also if the user entered text each user may use different words to describe the same thing. E.g. one user may say "Session not running" and another may say	The JLabel is bold and size 20 to make it easy to read. The text of the buttons are size 18 to make it easy to read.	46

“Session not on” and so the system would have to be able to interpret and understand these two different things to mean the same thing. Using buttons avoids this complicated programming. A combobox could be used but that would be slower than buttons.



THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Add label dialog	This allows the admin to specify the text that they want the label that they are adding to the question to have.	JLabel - “What should the text of the label be?” - This indicates to the admin the purpose of the JTextField. JTextField - This allows the admin to enter any text that they want the label to have. This is preferable compared to fixed options because it allows the questions that the system can create to be much more customisable. JButton - “Cancel” and “Ok” - These buttons allow the admin to cancel or finish the process of adding a label to the question.	The JLabel's text is size 20 and bold so that it's easy to read. The “Ok” and “Cancel” buttons are coloured to make their actions immediately obvious. Their text is bold and size 18 to make it easy to read.	2

Add text field

What type of validation is required?

Email

Cancel

ok

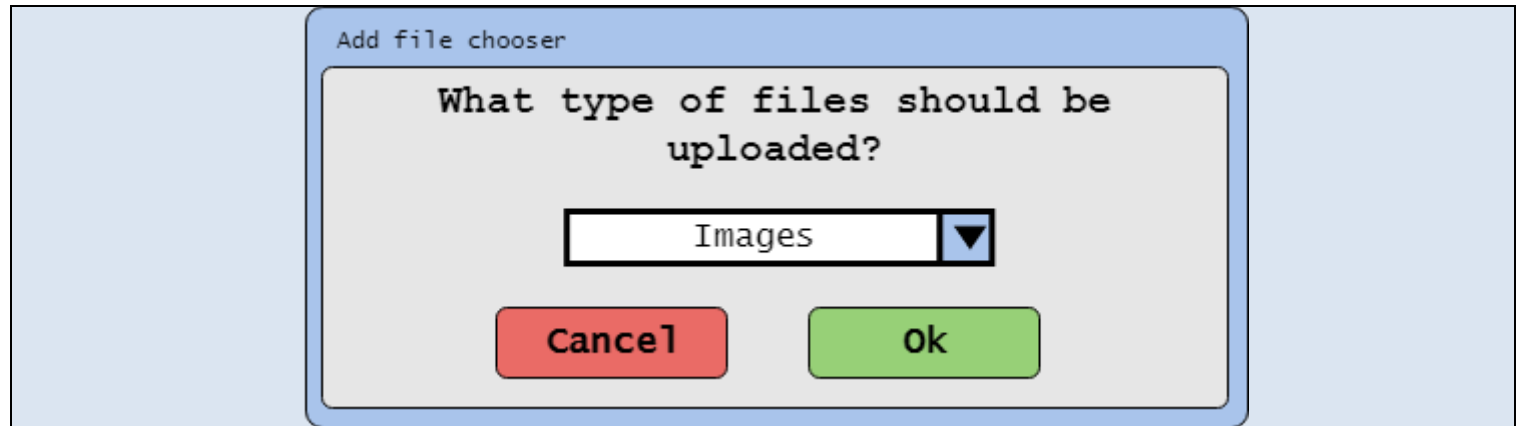
THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Add text field	This allows the admin to specify the type of data that will be entered into the text field. This means that the text field will have the appropriate validation applied to it.	JLabel - "What type of validation is required?" - This indicates to the admin the purpose of the JComboBox. JComboBox - This allows the admin to specify what type of validation is required. A JComboBox is preferable for this because it means that the admin can only select from fixed options. This is important because the system only supports certain types of validation. JButton - "Cancel" and "Ok" - These buttons allow the admin to cancel or finish the process of adding a text field to the question.	The JLabel's text is size 20 and bold so that it's easy to read. The "Ok" and "Cancel" buttons are coloured to make their actions immediately obvious. Their text is bold and size 18 to make it easy to read.	3, 4

THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Add options dialog	This allows the admin to specify the options required for any of the following types of components: Radio buttons, Checkboxes, Drop-down (ComboBox)	JLabel - "What options are required?" - This indicates to the admin the purpose of the dialog. JTable - This is used to clearly display the options that the admin has enter so far A table is better than using just a label because it allows individual options to be selected it also allows the text of the options to be edited by double clicking on a cell. JScrollPane - This is used to allow the table to be scrolled when there are too many options to view all at once on the screen. JLabel - "Option:" - This indicates the purpose of the JTextField to its right to the admin JTextField - This allows the admin to enter a new option. JButton - "Add option" - This allows the admin to add the option that has been entered into the JTextField to the list of options. JButton - "Delete selected" - This allows the admin to delete the currently selected option in the JTable.	The text of the JLabels is size 20 and bold so that it's easy to read. Text of the JButtons is size 18 that it is easy to read and can fit in the limited space of the buttons. The "Ok" and "Cancel" buttons are coloured to make their actions immediately obvious. Their text is bold and size 18 to make it easy to read.	5,6,7

		JButton - "Cancel" and "Ok" - These buttons allow the admin to cancel or finish the process of adding a text field to the question.		
--	--	--	--	--



THIS DESIGN WAS CREATED IN DRAW.IO AND IS NOT AN IMPLEMENTATION

Name of Design	Description of Design & Purpose	Components Used & Justification	Font: Type, Colour, Size	Objective #'s met
Add file chooser dialog	This allows the admin to specify the types of files that should be allowed to be uploaded using the filechooser and ensures that the appropriate validation is applied to the uploaded files.	JLabel - "What type of files should be uploaded?" - This indicates to the admin the purpose of the JComboBox. JComboBox - This allows the admin to specify what type of validation is required. A JComboBox is preferable for this because it means that the admin can only select from fixed options. This is important because the system only supports certain types of validation. JButton - "Cancel" and "Ok" - These buttons allow the admin to cancel or finish the process of adding a text field to the question.	The JLabel's text is size 20 and bold so that it's easy to read. The "Ok" and "Cancel" buttons are coloured to make their actions immediately obvious. Their text is bold and size 18 to make it easy to read.	10, 20

3.3 Files & Data Structures

3.31 Methods of Access inc. Files

Entity	Classes	Expected # of Records	Storage (Files & Arrays)	Method of access	Justification for Method of Access
Form	<i>Form.java va FormList.java</i>	30+	<i>FormDB.txt is a permanent storage of the Forms. FormList will temporarily store the Forms while the system is running that they can be quickly accessed from RAM.</i>	<i>Sequen tial</i>	● There is a relatively small number of records therefore the slower speed of serial access won't be an issue. ● There are many different sorts that can be applied by the user to the list of Forms therefore storing the data sequentially is of little use as it's likely that the user will reorder them anyway by applying a sort. ● Forms can be searched for by ID. If the Forms were in ID order a binary search could be used and this would speed up the search. However, for the reasons above even if the Forms were stored in key field over it's unlikely that they would remain that way throughout the program running. Therefore, if a binary search was to be performed they would have to be reordered back into key field order and this additional processing time would negate the faster search time of a binary search compared to a linear search. ● The Forms will be accessed every time that a Form is searched for or opened to be attempted. This means that the Forms will be accessed every 20 minutes depending on how quickly the user completes a Form and moves onto the next one. ● Sequential would take more time to program and would provide little benefit as the array would be reordered often as sorts are often applied to it. Random access is unnecessary because of how

					few records there are and so the speed improvement would not warrant the additional programming time and complexity of the system. ● Serial organisation will require the least programming time out of all the methods and will be simple to program.
Question	Question.java QuestionList.java	80+	QuestionDB.txt will permanently store the questions. QuestionList will temporarily store the questions while the system is running that they can be quickly accessed from RAM. TypeDB.txt will permanently store the possible types that the questions can be e.g. (email, dropdown, etc.). A string[] that is an attribute of QuestionList will temporarily store the types while the program is running so that they can be quickly accessed from RAM.	Serial	● There are a large number of Questions and this number will increase as Steph adds new Questions to the system. ● There are many different sorts that can be applied by the user to the list of Forms therefore storing the data sequentially is of little use as it's likely that the user will reorder them anyway by applying a sort. ● There are many different sorts (such as by difficulty and by type) that can be applied by the user to the list of Questions therefore storing the data sequentially is of little use as it's likely that the user will reorder them anyway by applying a sort. ● The Questions are accessed when the list of questions is displayed to the user and when the list of form is displayed to the user. This is because information about each question needs to be retrieved by the program in order to be displayed to the user. For example, the type and difficulty of the question. ● The main operation that is performed by the QuestionList is searching for a Question by ID (this happens when the forms or questions are displayed to the user). Because the ID's are numeric a binary search could be applied to increase the speed of the search. A sequential ordering allows this type of

					search to be performed. This means that it's beneficial to store the Questions sequentially ordered by their ID so that they can be looked up (and retrieved) more quickly by the system. Increasing the speed of this crucial and common operation will speed up the system. However, the disadvantage of storing the questions sequentially is that sorts are often applied to the question list therefore it's unlikely that the questions would stay in key sequence order. This means that before a search is to be performed the questions would have to be reordered, and this time taken to reorder the questions would negate the speed improvement that storing them sequentially provides. Due to the relatively small number of questions random access would take longer to implement to warrant the small speed improvement that it would provide. ● Serial programming is faster to implement. Both sequential and random access would take longer to implement and this additional programming time wouldn't warrant the speed improvement as mentioned above
Question Panel	QuestionPanel.java QuestionPanelList.java	80+ (same number of records as question)	QuestionPanelDB.txt will permanently store the questionPanels. QuestionPanelList will temporarily store the questionPanels while the system is running that they can be quickly accessed from RAM.	Sequential	● There are a large number of QuestionPanels and this number will increase as Steph adds new Questions to the system. ● The QuestionPanels are never reordered therefore they would remain in key field order. (Sorted by ID) ● The QuestionPanels are accessed every time that a Form is attempted or a Question is attempted. This is because the QuestionPanel

					corresponding to the Question or each Question in the Form needs to be retrieved so that it can be displayed to the user. There can be any number of Questions in a Form, usually around 10, and this means that the QuestionPanelList is accessed multiple times when attempting a Form. ● The main operation that is performed by the QuestionPanelList is searching for a QuestionPanel by ID (this happens when a Form or Question is loaded to be attempted). Because the ID's are numeric a binary search could be applied to increase the speed of the search. A sequential ordering allows this type of search to be performed. This means that it's beneficial to store the QuestionPanels sequentially ordered by their ID so that they can be looked up (and retrieved) more quickly by the system. Increasing the speed of this crucial and common operation will speed up the system a large amount. ● Serial would be easier to program and implement however it would mean that in order for a binary search to be performed rather than a slower linear search the records would have to be sorted before the search and this would be slower than storing the records sequentially and keeping them in order all of the time. Using random access would be fastest however the complexity and additional time taken to program outweighs its advantages. ● Sequential will require a moderate amount of programming time as it's
--	--	--	--	--	--

					more complex than serial however I think that the speed improvement that it would warrant the speed improvement that it would provide.
User	User.java a UserList. java	50+ for the admin's instance of the system 2-3 for the user's instance of the system	UserDB.txt will permanently store the data about all of the users and will be encrypted. UserList temporarily store the users while the program is running so that they can be quickly accessed from RAM.	Serial	- There is a varying number of these records based on whether the system is used by the admin or by a regular user. This is 2-3, for the user, and 50+ for the admin. There is only 1 admin instance of the system and around 10 user instances of the system therefore optimising this data structure for the regular users is more important than optimising for the admin. - For a user's instance of the system, Users are searched for by username when logging in to the system to validate their credentials. No further searches are performed. For the admin's instance of the system, Users can be searched for via first name. Users are also searched for via ID when logging their attendance. More searches are performed by the admin's instance of the system. As the User IDs are numeric a binary search could be performed to speed up the search by ID however the register is only taken once per session therefore as the operation is only performed once per session it doesn't justify the additional programming time required to speed up this operation of the system. - The User database is loaded into RAM when the program starts up. It is then accessed to validate the user's login details. After that, for the user's instance of the system, it is not accessed again. For

					the admin's instance of the system, it is accessed when the admin wants to take a register, view details about a user, or search for a user. As mentioned above, the admin will take a register once per session. It's unlikely that the admin will search for a user or view details about a user more than 2-3 times per session as the information stored isn't relevant to teaching the users how to fill in forms as the information stored is the users' phone numbers, for example. As these actions are performed so infrequently a faster method of access isn't required. ● The disadvantages of using sequential storage is that it would take longer to program than serial and would make the system more complex. Also as there are three different types of searches performed (by first name, by username, by ID) sequential would only speed up one of those searches at a time and the others would either have to use a slower linear search or the records would have to be reordered. Because there are so few records (2-3 for a user's instance) a linear search would be faster than resorting and then performing a binary search. ● Random access is unnecessary because there are so few records that the speed improvement would be negligible and would not warrant the additional programming time. ● Serial requires the least programming time out of the 3 options as it is the least complex and due to the reasons above the extra
--	--	--	--	--	--

					programming time required to implement sequential or random access would not warrant the speed increase that they would provide.
Question Stat	QuestionStat.java QuestionStatList.java	160+ (the number of questions multiplied by the number of users using the instance of the system) 0 for the admin's instance.	QuestionStatDB.txt will permanently store the QuestionStats belonging to a particular user. There will be a folder for each user and each folder will have the corresponding user's database. QuestionStatList will temporarily store the QuestionStats while the program is running so that they can be quickly accessed from RAM.	Random access	● There are a fairly large number of records to access. Using random access means that each record can be accessed in the same amount of time regardless of its position in the array. This means that access will remain fast even if the database grows to a much larger size. The database will grow in the future as Steph adds more questions to the system therefore using Random access allows the system to be more future proof as it will be able to have many more questions added to it without slowing down. ● The only search performed on this data is searching for a QuestionStat by ID. Because the only search is retrieving a QuestionStat with a particular ID random access is well suited to be used as the ID can be hashed to determine where to store the record. ● Every time that a question is validated it's QuestionStat is retrieved and updated. As there will usually be around 10-15 Questions in a Form just one validation of the Form will cause 10-15 lookups in the QuestionStat database. It's likely that it will take the user multiple attempts to correctly complete a Form therefore in one Form attempt the QuestionStat database could be accessed a large number of times (the number of questions in the form multiplied by the number of times the form was validated) this could be in the range of 50+. Because the database is

					accessed so often increasing the speed that the system can access a <code>QuestionStat</code> from the database will provide a large speedup to the system. ● Serial would be too slow because of how many records there are to access and how often they are accessed. Sequential would be faster because a binary search could be employed however it would become slower as the database grows and so would not be consistently fast like random access. ● Programming random access will take the longest potentially a few hours because I have not implemented random access before.
Form in progress	FormInProgress.java FormInProgressList.java	4+ (The number of users that the instance of the system has multiplied by 2 as it's likely users will only have 2 forms in progress at a time.) 0 for the admin's instance.	FormInProgress objects will temporarily store a copy of the question panels that the user is in the process of completing. FormInProgressList will temporarily store the FormInProgressObjects while the system is running. FormsInProgressDB.txt will permanently store the FormInProgress objects.	Serial	● There are a very small number of records to access and so a method of access faster than serial is not required as access via serial would be instantaneous. ● No sorts will be applied on the list of FormsInProgress. Searching for a FormInProgress by ID will be the only search operation that is performed on the list. This would be a linear search and because there will only be a very small number of elements the search will be able to be performed very quickly. ● The FormInProgress database is loaded into RAM when the program starts up. It is then accessed when the user continues attempting a form that they have previously attempted. ● Both sequential and random access would introduce more complexity into the program and would negligibly speed up the search operation. For this reason, serial access is better because it is much

					<i>simpler and faster to program than the other two methods of access. It is unlikely that the number of forms in progress would increase to be much more than 4 throughout the system's use.</i>
--	--	--	--	--	---

3.33 Data Structure Designs

Form.java

Field Name	Key Field	Data Type	Length	Example Data	Validate?
id	<i>PK</i>	<i>String</i>	<i>9</i>	<i>F86189391</i>	
questions	<i>FK</i>	<i>String[]</i>	<i>2-10+ Each String has length 9</i>	<i>Q72529010,Q35477436,Q51308304,Q81533244</i>	
description	-	String	100+	Example blue badge form	
mainSkillsTested	-	String[]	Size varies with number of question in the Form. Each type is of length 3-10+	Name,Drop-down,Birthday,Address	
difficulty	-	int	2	4	

Question.java

Field Name	Key Field	Data Type	Length	Example Data	Validate?
id	<i>PK</i>	<i>String</i>	<i>9</i>	<i>Q72529010</i>	
difficulty	-	int	2	3	
type	-	String	3-10+	Drop-down	

QuestionPanel.java

Field Name	Key Field	Data Type	Length	Example Data	Validate?
questionID	<i>FK</i>	<i>String</i>	<i>9</i>	<i>Q72529010</i>	
components	-	Component Array[]	2-10	[<Object of type textfield>,<Object of type passwordfield>,<Object of type combobox>]	

User.java

Field Name	Key Field	Data Type	Length	Example Data	Validate?
id	<i>PK</i>	<i>String</i>	<i>9</i>	<i>U54125754</i>	
username	-	String	2-15+	fmaccourt1	
password	-	String	8-15+	pOolwgp6	This field is not validated to improve the ease of use of the system and make it less likely

					that users will forget their passwords. Strong passwords are not important as user accounts do not store sensitive information
firstName	-	String	2-15+	Forster	
lastName	-	String	2-15+	MacCourt	
dateOfBirth	-	String	10	29/04/1987	
phoneNumber	-	String	10-11	2478200469	
sessionsPresentAt	-	String[]	1-10+	[2018-07-22,2018-08-01]	
questionStats	-	QuestionStatList	60-80+	See Question Stat	

QuestionStat.java

Field Name	Key Field	Data Type	Length	Example Data	Validate?
questionID	FK	String	9	Q72529010	
timesFailedValidation	-	int	3	102	
numberOfAttemptsNeededToCorrect	-	int[]	5+	1,5,3,2,6	
timeTakenToComplete	-	long[]	5+	5,4,7,12,9	

FormInProgress.java

Field Name	Key Field	Data Type	Length	Example Data	Validate?
formID	FK	String	9	F86189391	
percentComplete	-	int	3	25	
questionPanels	-	questionPanel[]	2-10+	[<Object of type QuestionPanel>, <Object of type QuestionPanel>, <Object of type QuestionPanel>, <Object of type QuestionPanel>]	

3.34 File Designs

File	Values to be stored	Example
FormDB.txt	ID,Questions,Description,MainSkillsTested,Difficulty	F07377236,Q64356881.Q41647821,This tests the file upload saving,Name.File upload,1
QuestionDB.txt	ID,Difficulty,Type	Q67918670,1,Radio-buttons
QuestionPanelDB.txt	questionID\$Component Component	Q14805999\$label:Address locationentry:
TypesDB.txt	types (csv)	Address,Email,Drop-down,Radio-buttons,Birthday,Name,File Upload,Photo ID,Calendar,Checkboxes,Password,File upload,Phone
UserDB.txt	ID,Username>Password,Firstname, Lastname,DateOfBirth,PhoneNumber,SessionsPresentAt Question Stats	U10560101,lruller2,dCfEQZA3jA,Leela,Ruller,19-03-1967,02386249416,false,06-12-2018.07-12-2018.17-12-2018.18-12-2018.19-12-2018 Q36849304,0,0.0.-1.-1.-1,2.2.-1.-1.-1,2
<user>/FormsInProgressDB.txt	FormID;PercentComplete~QuestionPanels QuestionPanels etc.	F69383296;75;0~Q64356881\$label:Name textfield:Harris;none,Q67918670\$label:Gender radiobuttons:Male>true.Female>false.Other>false,Q84848110\$label:Email Address textfield;;email,header:More about you,Q71119615\$label:Birthda y datepicker:2019-01-15

3.4 Validation

#	Class	Field	Val. Type	Val. Rule/Description	Error Message
1	User.java	Username	Lookup	Check that the username entered isn't already in use by a different user	Username taken.
2	User.java	Date of Birth	Format	Check that the date is in the form DD-MM-YYYY	Date is in the incorrect format. Should be DD-MM-YYYY
3	User.java	Phone Number	Length	Check that the phone number is no longer than 11 digits as this the longest UK phone number	Phone number too long. Please enter a number that is at most 11 digits long.
1	LoginWindow.java	Username	Presence	A check to ensure that the user has filled in the username field.	Please fill in the username field
2	LoginWindow.java	Password	Presence	A check to ensure that the user has filled in the password field.	Please fill in the password field
1	Form.java	Description	Presence	Check that the description of the form has been filled in by the admin.	Please enter a description for the form.
2	Form.java	Question array	Length	Check that the length is at least 1 - to check that the form contains at least 1 question.	Please add at least 1 question to the form.
1	JValidatedTextField.java	Text field	Presence	Check that the user has entered some text into the text field.	[Highlighted visually with red border] Text field empty
2	JValidatedTextField.java	Text field	Format	Check that the email address is in the format (and so valid) <String>@<String>.<String> or any other valid email format e.g. <String>.<String>@<String>.<String>.<String> but not <String>@.<String>	[Highlighted visually with red border] Email address invalid
3	JValidatedTextField.java	Text field	Length	Check that the phone number is no longer than 11 digits as this the longest UK phone number	[Highlighted visually with red border] Phone number invalid
1	JValidatedPasswordField.java	Password Field	Presence	Check that both the password fields are filled in	[Highlighted visually with red border] Both password fields are not filled in.
1	JValidatedRadioButtons.java	Radio buttons	Presence	Check that a radio button is checked	[Highlighted visually with red border]

					An option is not selected
1	JValidatedComboBox.java	Combobox	Presence	Check that an option other than the "Please select an option" option is selected.	[Highlighted visually with a red border] An option is not selected
1	JValidatedCheckBoxes.java	Check Boxes	Presence	Check that at least one checkbox is checked	[Highlighted visually with a red border] An option is not selected.
1	JValidatedDatePicker.java	Date	Presence	Check that a date has been entered into the date text field.	[Highlighted visually with a red border] Date not entered
2	JValidatedDatePicker.java	Date	Format	Check that the date is in the format DD-MM-YYYY	[Highlighted visually with a red border] Date not in DD-MM-YYYY
1	JValidatedFileChooser.java	File	Presence	Check that a file has been selected to be uploaded	[Highlighted visually with a red border] File not selected
2	JValidatedFileChooser.java	File	Lookup	Check that the type of the file uploaded is of the types that the uploader allows. E.g. if the question was to upload a picture then only files which have types that are images would be allowed e.g. .png and .jpg .	[Highlighted visually with a red border] File of incorrect type
1	QuestionPanel.java	Components array	Length	Check that the length of the array is at least 1 showing that at least 1 component has been added to the question	Please add some components to the question

3.5 Processes

3.51 Objectives Links with Data/Files

Objective # (Investigation)	Identification of Process	Data/Files Used	Explanation of Process
1	Create Question	Data: id, difficulty, type, components (in the question) Files: questionDB, typesDB, questionPanelDB	Allows the admin to create a question by selecting the components (the actual gui elements) that they want to be part of the question, and also entering the difficulty and type of the question.
2	Add labels	Data: text	Allows the admin to specify the text of a label and add the label to a question.
	Save labels	Data: text Files: formInProgressDB	Allows the label to be saved to file with the text preserved.
	Load labels	Data: text Files: formInProgressDB	Allows the label to be loaded from file with the text preserved.
3	Add text fields	Data: validation required	Allows the admin to specify the type of validation a text field requires (e.g. validate phone numbers) and add the text field to a question
	Save text fields	Data: text, validation required Files: formInProgressDB	Allows the textfield to be saved to file with the entered text, and the type of validation preserved.
	Load text fields	Data: text, validation required Files: formInProgressDB	Allows the textfield to be loaded from file with the entered text, and the type of validation preserved.
5	Add radio buttons	Data: options	Allows the options that can be selected using radio buttons to be specified, by the admin, and the group of radio buttons to be added to a question.
	Save radio buttons	Data: options, selected option Files: formInProgressDB	Allows the radio buttons to be saved to file with the selected option, and all other options preserved.
	Load radio buttons	Data: text,	Allows the radio buttons to be loaded from file with the selected option, and all other

		validation required Files: formInProgressDB	options preserved.
6	Add drop-downs	Data: options	Allows the options that can be selected using a drop-down to be specified, by the admin, and the drop-down to be added to a question.
	Save drop-downs	Data: options, selected option Files: formInProgressDB	Allows the drop-down to be saved to file with the selected option, and all other options preserved.
	Load drop-downs	Data: text, validation required Files: formInProgressDB	Allows the drop-down to be loaded from file with the selected option, and all other options preserved.
7	Add check boxes	Data: options	Allows the options that can be selected using check boxes to be specified, by the admin, and the group of check boxes to be added to a question.
	Save check boxes	Data: options, selected option Files: formInProgressDB	Allows the check boxes to be saved to file with the selected option, and all other options preserved.
	Load check boxes	Data: text, validation required Files: formInProgressDB	Allows the check boxes to be loaded from file with the selected option, and all other options preserved.
8	Add password fields	-	Allows password fields to be added to a question by the admin.
	Save password fields	Data: password1, password2 Files: formInProgressDB	Allows password fields to be saved to file with the passwords entered in the fields preserved.
	Load password fields	Data: password1, password2 Files: formInProgressDB	Allows password fields to be loaded from file with the passwords entered in the fields preserved.
9	Add calendar entry	-	Allows the admin to add a date picker to a question
	Save calendar entry	Data: date selected	Allows date pickers to be saved to file with the selected date preserved.

		Files: formInProgressDB	
	Load calendar entry	Data: date selected Files: formInProgressDB	Allows date pickers to be loaded from file with the selected date preserved.
10	Add document upload field	Data: validation required	Allows the admin to add a file chooser to a question and specify the types of files that the file chooser should accept.
	Save document upload field	Data: validation required, selected file path Files: formInProgressDB	Allows date pickers to be saved to file with the validation required, and selected file path preserved.
	Load document upload field	Data: validation required, selected file path Files: fromInProgressDB	Allows date pickers to be loaded from file with the validation required, and selected file path preserved.
11	Delete questions	Data: questionID Files: questionDB, questionPanelDB	Allows the admin to delete a question from the system by specifying its id.
12	Sort questions by difficulty	Data: questionArray, difficulty (of each question)	Allows the user to sort the questions by difficulty.
13	Sort questions by type	Data: questionArray, type (of each question)	Allows the user to sort the questions alphabetically by type.
14	Filter questions by difficulty	Data: questionArray, difficulty (of each question)	Allows the user to filter the questions by difficulty.
15	Filter questions by type	Data: questionArray, type (of each question)	Allows the user to filter the questions alphabetically by type.
16	Format check email addresses, dates, and phone numbers	Data: email address, date, phone number	A format check is performed on an entered email address, date or phone number and the outcome is returned and used to determine whether the user has correctly filled in the question.
18	Double entry verification on passwords	Data: password1, password2	password1 and password2 are checked to see if they match and the outcome is returned and used to determine whether the user has correctly filled in the question.
19	Presence check on all	Data:	The input that the user is checked to see

	fields	text entered, selected option, selected file path etc.	whether it is empty for not / whether they have not selected an option yet and the outcome is returned and used to determine whether the user has correctly filled in the question.
20	Asserting that the selected file in a document upload field is a valid type	Data: selected file path	The extension of the selected file is checked to ensure that it is of one of the allowed types e.g. for a document upload field that only allows photos to be selected the extension is checked to ensure that it is one of .png, .jpg, or .bmp
21	Export question to file	Data: id, difficulty, type, question panel (components) Files: exportedQuestion (creates a new file)	Allows the admin to export a question and all of its attributes and components to a file.
22	Import question from file	Data: id, difficulty, type, question panel (components) Files: exportedQuestion	Allows the admin/user to import a question and all of its attributes and components from a file. This allows the transfer of questions between different instances of the system.
25	Create forms	Data: id, questions (a list of the ids of the questions that are in the form), description, main skills tested (a list of all of the types of the questions that are in the form), difficulty. Files: formDB	Allows the admin to create a form by selecting the questions that they want to be part of the form, and also entering a description of the form and its difficulty.
26	Create form from user weaknesses	Data: question stats (all of the question stat objects corresponding to questions that the user has attempted) , average number of attempts needed to correct an error (for each question stat)	Creates a form for the user to attempt which contains the top 5 questions that the user takes the most attempts to correct an error on.
27	Sort forms by difficulty	Data: formArray, difficulty (of each	Allows the user to sort the forms by difficulty.

		form)	
28	Filter forms by difficulty	Data: formArray, difficulty (of each form)	Allows the user to filter the forms by difficulty.
29	Filter forms by type of questions tested	Data: formArray, type (of each question in each form)	Allows the user to filter forms so that only forms that contain certain types of question remain.
30	Edit forms	Data: id, questions (a list of the ids of the questions that are in the form), description, main skills tested (a list of all of the types of the questions that are in the form), difficulty. Files: formDB	Allows the admin to edit the form's description and difficulty. They are also able to remove and add questions to the form and change the order of the questions in the form.
31	Delete forms	Data: id Files: formDB	Allows the admin to delete a form from the system by specifying the id of the form that they want to delete.
32	Export form to file	Data: id, questions (a list of the ids of the questions that are in the form), description, main skills tested (a list of all of the types of the questions that are in the form), difficulty. Files: exportedForm (creates new file)	Allows the admin to export a form and all of its attributes and questions to a file.
33	Import form from file	Data: id, questions (a list of the ids of the questions that are in the form), description, main skills tested (a list of all of the types of the questions that are in the form), difficulty. questionArray (an array containing	Allows the admin/user to import a form and all of its attributes and questions from a file without creating duplicate questions in the system as before a question is loaded it's checked to ensure that it doesn't already exist in the instance of the system. This allows the transfer of forms between different instances of the system.

		all of the questions currently in the system) Files: exportedForm	
35	Submit a form	Data: questions (the questions that are in the form and the data about how they are filled in e.g. the contents of each field), current logged in User (the logged in users account), question stat list (a list of all the question stat objects belonging to a user), times failed validation (for each question), time taken to complete (for each question) Files: questionStatDB (belonging to the user)	Each question in the form is validated and then the user's question stat objects are appropriately updated. E.g. the number of times that they have failed the validation check is incremented to match with the number of times that they failed completing the form.
36	Tracking time taken to complete question	Data: time question gained focus, time question lost focus, total time (for the current attempt of the question)	The time that it has taken a user to complete a question is calculated by working out the duration of time that the user has had the question focused and then adding that time to a running total for the current attempt of the form.
37	Tracking the number of times the user has failed the validation check for each question	Data: failed validation (boolean), total number of times failed validation (for the current attempt of the question), total number of times failed validation (overall)	When the questions are validated, the questions that the user has failed the validation check of have their number of times the user has failed validation total incremented for both the current attempt of the question and the overall total.
38	Add users	Data: id, username, password, first name, last name, date of birth, phone number, sessions present at (a list of the dates	Allows the admin to add new users to the system by entering the personal details of the user.

		of the sessions that the user has attended) Files: userDB	
39	Edit users	Data: id, username, password, first name, last name, date of birth, phone number, sessions present at (a list of the dates of the sessions that the user has attended) Files: userDB	Allows the admin to edit the data stored about each user. (e.g. their phone number)
40	Delete users	Data: id, userArray Files: userDB	Allows the admin to delete a user from the system by specifying the id of the user to delete.
41	Search through users	Data: id	Allows a specific user to be retrieved from the system by id.
42	Produce a report on the user's weaknesses/progress	Data: user id (id of the user to produce a report of), question stat list (belonging to the user), time taken to complete array (for each question), number of attempts taken to correct an error array (for each question), total number of times failed validation (for each question)	Allows the admin/user to produce and print a report that details the user's progress / weaknesses / progress with each question in the system.
43	Tracking how much of a form the user has attempted	Data: question array (a list of all the questions in the form), failed validation (whether the user has failed the validation for each question in the form)	The system automatically calculates the percentage that the user has completed of each form by dividing the number of questions that they have passed the validation check by the total number of questions that are in the form and they multiplying by 100.

		Files: formInProgressDB	
45	Attendance tracking for each user	Data: current date, sessions present at array (for the user) Files: userDB	The system automatically logs the attendance of each user when they log in by appending the current date to the user's sessions present at array.
46	Automatically determine a reason why the user has missed a session	Data: most recent session date, sessions present at array (for the user), response (the user's answer to the questions which asks them why they missed the last session)	The system automatically determines whether a user missed the last session by calculating the date of the last session and then checking to see whether that date is in the user's sessions present at array. If it is not in the user's sessions present at array the system allows the user to select the reason that they missed the session from a list.
63	Ability to take a register	Data: current date, user array (list of all the users), sessions present at array (for each user) Files: userDB	Allows the admin to take a register by marking which users are present at the current session using check boxes.

3.52 Processing Routines

// Objective 1 Ability to create Questions

START createQuestion()

GLOBAL ARRAY OF TYPE QUESTION questionArray IS PREDECLARED {has been filled with questions from file}

GLOBAL VARIABLE OF TYPE INTEGER nextQuestionLocation IS PREDECLARED {Next free location in the array}

DECLARE newQuestion IS NEW Question()

STRING ATTRIBUTE id IS PREDECLARED IN QUESTION class

STRING ATTRIBUTE difficulty IS PREDECLARED IN QUESTION class

STRING ATTRIBUTE type IS PREDECLARED IN QUESTION class

DECLARE newId IS STRING

SET newId = CALL METHOD getFreeID() {getFreeID() is predeclared}

INPUT newDifficulty

INPUT newType

SET newQuestion.id = newId

SET newQuestion.difficulty = newDifficulty

SET newQuestion.type = newType

createQuestionPanel(newID)

SET questionArray[nextQuestionLocation] = newQuestion

CALL METHOD writeDatabase()

END

START createQuestionPanel(Parameter: id)

GLOBAL ARRAY OF TYPE QUESTIONPANEL questionPanelArray IS PREDECLARED {has been filled with questionPanels from file}

GLOBAL VARIABLE OF TYPE INTEGER nextQuestionPanelLocation IS PREDECLARED {Next free location in the array}

DECLARE newQuestionPanel IS NEW QuestionPanel()

STRING ATTRIBUTE questionID IS PREDECLARED IN QUESTIONPANEL class

STRING[] ATTRIBUTE components IS PREDECLARED IN QUESTIONPANEL class

INPUT newComponents {A list of all the components that make up a questionPanel}

SET newQuestionPanel.questionID = id

SET newQuestionPanel.components = newComponents

SET questionPanelArray[nextQuestionPanelLocation] = newQuestionPanel

END

// Objective 2 Ability to add labels

START addLabel()

DECLARE newLabel IS NEW Label()

STRING ATTRIBUTE text IS PREDECLARED IN LABEL class

```

    INPUT labelText

    SET newLabel.text = labelText
END

START save() {method of class label}
    STRING ATTRIBUTE text IS PREDECLARED IN LABEL class

    OUTPUT "label:" + text
END

START loadLabel(Parameter: saveString)
    DECLARE loadedLabel IS NEW Label()
    STRING ATTRIBUTE text IS PREDECLARED IN LABEL class

    SET loadedLabel.text = saveString[0]
END

```

// Objective 3 Ability to add text fields

```

START addTextField()
    DECLARE newTextField IS NEW TextField()
    STRING ATTRIBUTE validationRequired IS PREDECLARED IN TEXTFIELD class

    INPUT newValidationRequired

    SET newTextField.validationRequired = newValidationRequired
END

START save() {method of class TextField}
    STRING ATTRIBUTE text IS PREDECLARED IN TEXTFIELD class {The text in the field}
    STRING ATTRIBUTE validationRequired IS PREDECLARED IN TEXTFIELD class

    OUTPUT "textfield:" + validationRequired + " | " + text
END

START loadTextField(Parameter: saveString)
    DECLARE loadedTextField IS NEW TextField()
    STRING ATTRIBUTE text IS PREDECLARED IN TEXTFIELD class
    STRING ATTRIBUTE validationRequired IS PREDECLARED IN TEXTFIELD class

    SET loadedTextField.validationRequired = saveString[0]
    SET loadedTextField.text = saveString[1]
    RETURN loadedTextField
END

```

// Objective 4 Ability to specify the type of data that should be entered into the text field

{In objective 3 - validationRequired attribute}

// Objective 5 Ability to add radio buttons

```

START addRadioButtons()
    DECLARE newRadioButtons IS NEW RadioButtons()
    STRING[] ATTRIBUTE options IS PREDECLARED IN RADIOBUTTONS class

    DECLARE newOptions IS String[100]
    DECLARE moreOptions IS BOOLEAN

```

```

SET moreOptions = TRUE
SET i = 0

WHILE moreOptions DO
    INPUT newOptions[i]
    OUTPUT "Continue entering options? "
    INPUT response
    IF response == "n" THEN {If they entered n (for no)}
        SET moreOptions = FALSE
    ENDIF
    SET i = i + 1
ENDWHILE

DECLARE trimmedOptions IS STRING ARRAY
DECLARE j is INTEGER
FOR j=0 TO i DO
    trimmedOptions[j] = newOptions[j]
ENDFOR

SET newRadioButtons.options = newOptions
END

START save() {method of class RadioButtons}
    STRING ATTRIBUTE options IS PREDECLARED IN RADIOBUTTONS class {The options that
    can be selected}
    STRING ATTRIBUTE selectedOption IS PREDECLARED IN RADIOBUTTONS class

    DECLARE optionsString IS STRING
    DECLARE i IS INTEGER
    SET i=0
    SET optionsString = options[i]
    FOR i=1 TO i=options.length DO
        SET optionsString = optionsString + "." + options[i]
    ENDFOR

    OUTPUT "radiobuttons:" + optionsString + "|" + selectedOption
END

START loadRadioButtons(Parameter: saveString)
    DECLARE loadedRadioButtons IS NEW RadioButtons()
    STRING ATTRIBUTE options IS PREDECLARED IN RADIOBUTTONS class
    STRING ATTRIBUTE selectedOption IS PREDECLARED IN RADIOBUTTONS class

    SET loadedRadioButtons.options = saveString[0].split(".")
    SET loadedRadioButtons.selectedOption = saveString[1]

    RETURN loadedRadioButtons
END

```

// Objective 6 Ability to add drop-downs

```

START addComboBox()
    DECLARE newComboBox IS NEW ComboBox()
    STRING[] ATTRIBUTE options IS PREDECLARED IN COMBOBOX class

    DECLARE newOptions IS String[100]

```

```

DECLARE moreOptions IS BOOLEAN
SET moreOptions = TRUE
SET i = 1
SET newOptions[0] = "Please select an option"

WHILE moreOptions DO
    INPUT newOptions[i]
    OUTPUT "Continue entering options? "
    INPUT response
    IF response == "n" THEN {If they entered n (for no)}
        SET moreOptions = FALSE
    ENDIF
    SET i = i + 1
ENDWHILE

DECLARE trimmedOptions IS STRING ARRAY
DECLARE j is INTEGER
FOR j=0 TO i DO
    trimmedOptions[j] = newOptions[j]
ENDFOR

SET newComboBox.options = newOptions
END

START save() {method of class ComboBox}
    STRING ATTRIBUTE options IS PREDECLARED IN ComboBox class {The options that can
    be selected}
    STRING ATTRIBUTE selectedOption IS PREDECLARED IN ComboBox class

    DECLARE optionsString IS STRING
    DELARE i IS INTEGER
    SET i=0
    SET optionsString = options[i]
    FOR i=1 TO i=options.length DO
        SET optionsString = optionsString + "." + options[i]
    ENDFOR

    OUTPUT "combobox:" + optionsString + "|" + selectedOption
END

START loadComboBox(Parameter: saveString)
    DECLARE loadedComboBox IS NEW ComboBox()
    STRING ATTRIBUTE options IS PREDECLARED IN COMBOBOX class
    STRING ATTRIBUTE selectedOption IS PREDECLARED IN COMBOBOX class

    SET loadedComboBox.options = saveString[0].split(".")
    SET loadedComboBox.selectedOption = saveString[1]

    RETURN loadedComboBox
END

// Objective 7 Ability to add checkboxes

START addCheckBoxes()
    DECLARE newCheckBoxes IS NEW CheckBoxes()
    STRING[] ATTRIBUTE options IS PREDECLARED IN CHECKBOXES class

```

```

DECLARE newOptions IS String[100]
DECLARE moreOptions IS BOOLEAN
SET moreOptions = TRUE
SET i = 0

WHILE moreOptions DO
    INPUT newOptions[i]
    OUTPUT "Continue entering options? "
    INPUT response
    IF response == "n" THEN {If they entered n (for no)}
        SET moreOptions = FALSE
    ENDIF
    SET i = i + 1
ENDWHILE

DECLARE trimmedOptions IS STRING ARRAY
DECLARE j is INTEGER
FOR j=0 TO i DO
    trimmedOptions[j] = newOptions[j]
ENDFOR

SET newCheckBoxes.options = newOptions
END

START save() {method of class CheckBoxes}
    STRING ATTRIBUTE options IS PREDECLARED IN CheckBoxes class {The options that can
    be selected}
    STRING ATTRIBUTE selectedOption IS PREDECLARED IN CheckBoxes class

    DECLARE optionsString IS STRING
    DELARE i IS INTEGER
    SET i=0
    SET optionsString = options[i]
    FOR i=1 TO i=options.length DO
        SET optionsString = optionsString + "." + options[i] + ";" options[i].isSelected()
    ENDFOR

    OUTPUT "checkboxes:" + optionsString + "|" + selectedOptions
END

START loadCheckBoxes(Parameter: saveString)
    DECLARE loadedCheckBoxes IS NEW CheckBoxes()
    STRING ATTRIBUTE options IS PREDECLARED IN CHECKBOXES class
    STRING ATTRIBUTE selectedOption IS PREDECLARED IN CHECKBOXES class

    SET loadedCheckBoxes.options = saveString[0].split(".")
    SET loadedCheckBoxes.selectedOption = saveString[1]

    RETURN loadedCheckBoxes
END

```

// Objective 8 Ability to add password fields

```

START addPasswordField()
    DECLARE newPasswordFields IS NEW PasswordFields() {No additonal data required)

```

END

START save() {method of class PasswordFields}
 STRING ATTRIBUTE password1 IS PREDECLARED IN PASSWORDFIELDS class
 STRING ATTRIBUTE password2 IS PREDECLARED IN PASSWORDFIELDS class

OUTPUT "passwordfields:" + password1 + "|" + password2

END

START loadPasswordFields(Parameter: saveString)
 DECLARE loadedPasswordFields IS NEW PasswordFields()
 STRING ATTRIBUTE password1 IS PREDECLARED IN PASSWORDFIELDS class
 STRING ATTRIBUTE password2 IS PREDECLARED IN PASSWORDFIELDS class

SET loadedPasswordFields.password1 = saveString[0]

SET loadedPasswordFields.password2 = saveString[1]

RETURN loadedPasswordFields

END

// Objective 9 Ability to add calendar entry

START addCalendarEntry()
 DECLARE newCalendarEntry IS NEW CalendarEntry() {No additional data required}

END

START save() {method of class CalendarEntry}
 STRING ATTRIBUTE dateSelected IS PREDECLARED IN CALENDARENTRY class

OUTPUT "calendarentry:" + dateSelected

END

START loadCalendarEntry(Parameter: saveString)
 DECLARE loadedCalendarEntry IS NEW CalendarEntry()
 STRING ATTRIBUTE dateSelected IS PREDECLARED IN CALENDARENTRY class

SET loadedCalendarEntry.dateSelected = saveString[0]

RETURN loadedCalendarEntry

END

// Objective 10 Ability to add a document upload field

START addDocumentUploadField()
 DECLARE newDocumentUploadField IS NEW DocumentUploadField()
 STRING ATTRIBUTE validationRequired IS PREDECLARED IN DOCUMENTUPLOADFIELD class

INPUT newValidationRequired

SET newDocumentUploadField.validationRequired = newValidationRequired

END

START save() {method of class DocumentUploadField}
 STRING ATTRIBUTE selectedFilePath IS PREDECLARED IN DocumentUploadField class
 {The path of the file that is selected}
 STRING ATTRIBUTE validationRequired IS PREDECLARED IN DOCUMENTUPLOADFIELD

```

class
    OUTPUT "documentuploadfield:" + validationRequired + "|" + selectedFilePath
END

START loadDocumentUploadField(Parameter: saveString)
    DECLARE loadedDocumentUploadField IS NEW DocumentUploadField()
    STRING ATTRIBUTE selectedFilePath IS PREDECLARED IN DocumentUploadField class
    {The path of the file that is selected}
    STRING ATTRIBUTE validationRequired IS PREDECLARED IN DOCUMENTUPLOADFIELD
    class

    SET loadedDocumentUploadField.validationRequired = saveString[0]
    SET loadedDocumentUploadField.selectedFilePath = saveString[1]

    RETURN loadedDocumentUploadField
END

// Objective 11 Ability to delete Questions
START deleteQuestion(Parameter: questionIDToDelete)
    GLOBAL ARRAY OF TYPE QUESTION questionArray IS PREDECLARED {has been filled
    with questions from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextQuestionLocation IS PREDECLARED {Next
    free location in the array}
    DECLARE newArray[questionArray.length] IS QUESTION[]
    DECLARE i IS INTEGER
    DECLARE j IS INTEGER

    SET j = 0

    FOR i=0 TO nextQuestionLocation DO
        IF (questionArray[i].ID != questionIDToDelete) THEN {If the question isn't the one
        that is to be deleted}
            SET newArray[j] = questionArray[i]
            SET j = j + 1
        ENDIF
    ENDFOR

    SET questionArray = newArray
    SET nextQuestionLocation = nextQuestionLocation - 1

    deleteQuestionPanel(questionIDToDelete)

    CALL METHOD writeDatabase()
END

START deleteQuestionPanel(Parameter: questionPanelIDToDelete)
    GLOBAL ARRAY OF TYPE QUESTIONPANEL questionPanelArray IS PREDECLARED {has
    been filled with questionPanels from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextQuestionPanelLocation IS PREDECLARED
    {Next free location in the array}
    DECLARE newArray[questionPanelArray.length] IS QUESTIONPANEL[]
    DECLARE i IS INTEGER
    DECLARE j IS INTEGER

    SET j = 0

```

```

FOR i=0 TO nextQuestionPanelLocation DO
    IF (questionPanelArray[i].ID != questionPanelIDToDelete) THEN {If the
        questionPanel isn't the one that is to be deleted}
        SET newArray[i] = questionPanelArray[i]
        SET j = j + 1
    ENDIF
ENDFOR

SET questionPanelArray = newArray
SET nextQuestionPanelLocation = nextQuestionPanelLocation - 1

CALL METHOD writeDatabase()
END

```

// Objective 12 Ability to sort Questions by difficulty

```

START
    GLOBAL ARRAY OF TYPE QUESTION questionArray IS PREDECLARED {has been filled
with questions from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextQuestionLocation IS PREDECLARED {Next
free location in the array}
    DECLARE i IS INTEGER
    DECLARE tempQuestion IS QUESTION
    DECLARE swapped IS BOOLEAN
    SET swapped = TRUE
    WHILE swapped == TRUE DO
        SET swapped = FALSE
        FOR i=0 TO nextQuestionLocation-1 DO
            IF (questionArray[i].difficulty > questionArray[i+1].difficulty) THEN
                SET tempQuestion = questionArray[i+1]
                SET questionArray[i+1] = questionArray[i]
                SET questionArray[i] = tempQuestion

                SET swapped = TRUE
            ENDIF
        ENDFOR
    ENDWHILE
END

```

// Objective 13 Ability to sort Questions by type

```

START compareWords(Parameter: word1, Parameter:word2) {Returns true if word1 comes
before word2 in the dictionary}
    DECLARE word1FirstLetter AS INTEGER
    DECLARE word2FirstLetter AS INTEGER
    DECLARE beforeInDictionary AS BOOL

    SET word1FirstLetter = (INT) word1[0] {The binary value of the first character of the
word}
    SET word2FirstLetter = (INT) word2[0] {The binary value of the first character of the
word}

    IF (word1FirstLetter < word2FirstLetter) THEN
        SET beforeInDictionary = TRUE
    ELSE IF (word1FirstLetter > word2FirstLetter) THEN
        SET beforeInDictionary = FALSE
    ENDIF
END

```



```

ELSE {They must be the same letter} DO
    IF (word1.length == 1 OR word2.length == 1) THEN
        IF (word1.length < word2.length) THEN
            SET beforeInDictionary = TRUE
        ELSE THEN
            SET beforeInDictionary = FALSE
        ENDIF
    ELSE THEN
        DECLARE trimmedWord1 as STRING
        DECLARE trimmedWord2 as STRING
        SET trimmedWord1 = word1.removeChar(0) {removeChar is predeclared
        and removes the first character of the word}
        SET trimmedWord2 = word1.removeChar(0)

        SET beforeInDictionary = compareWords(trimmedWord1, trimmedWord2)
        {Recursive call}
    ENDIF
ENDIF

RETURN beforeInDictionary
END compareWords

START
GLOBAL ARRAY OF TYPE QUESTION questionArray IS PREDECLARED {has been filled
with questions from file}
GLOBAL VARIABLE nextQuestionLocation OF TYPE INTEGER IS PREDECLARED {Next
free location in the array}
DECLARE i IS INTEGER
DECLARE tempQuestion IS QUESTION
DECLARE swapped IS BOOLEAN
SET swapped = TRUE
WHILE swapped == TRUE DO
    SET swapped = FALSE
    FOR i=0 TO nextQuestionLocation-1 DO
        IF (!compareWords(questionArray[i].type, questionArray[i+1].type))
        THEN {If questionArray[i]'s type doesn't compare before
        questionArray[i+1]'s type in the dictionary}
            SET tempQuestion = questionArray[i+1]
            SET questionArray[i+1] = questionArray[i]
            SET questionArray[i] = tempQuestion
        ENDIF
    ENDFOR
    SET swapped = TRUE
ENDWHILE
END

```

// Objective 14 Ability to filter Questions by difficulty

```

START filterByDifficulty(Parameter: filterDifficulty)
GLOBAL ARRAY OF TYPE QUESTION questionArray IS PREDECLARED {has been filled
with questions from file}
GLOBAL VARIABLE OF TYPE INTEGER nextQuestionLocation IS PREDECLARED {Next
free location in the array}
DECLARE i IS INTEGER
DECLARE results[nextQuestionLocation] IS QUESTION[]

```

```

DECLARE nextResultLocation IS INTEGER

SET nextResultLocation = 0

FOR i=0 TO nextQuestionLocation DO
    IF (questionArray[i].difficulty == filterDifficulty) THEN
        SET results[nextResultLocation] = questionArray[i]
        SET nextResultLocation = nextResultLocation + 1
    ENDIF
ENDFOR

DECLARE trimmedResults[nextResultLocation] IS QUESTION[]

FOR i=0 TO nextResultLocation DO
    SET trimmedResults[i] = results[i]
ENDFOR

RETURN trimmedResults
END

```

// Objective 15 Ability to filter Questions by type

```

START filterByType(Parameter: filterType)
    GLOBAL ARRAY OF TYPE QUESTION questionArray IS PREDECLARED {has been filled
    with questions from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextQuestionLocation IS PREDECLARED {Next
    free location in the array}
    DECLARE i IS INTEGER
    DECLARE results[nextQuestionLocation] IS QUESTION[]
    DECLARE nextResultLocation IS INT

    SET nextResultLocation = 0

    FOR i=0 TO nextQuestionLocation DO
        IF (questionArray[i].type == filterType) THEN
            SET results[nextResultLocation] = questionArray[i]
            SET nextResultLocation = nextResultLocation + 1
        ENDIF
    ENDFOR

    DECLARE trimmedResults[nextResultLocation] IS QUESTION[]

    FOR i=0 TO nextResultLocation DO
        SET trimmedResults[i] = results[i]
    ENDFOR

    RETURN trimmedResults
END

```

// Objective 16 Email addresses, dates, and phone numbers should have format checks

```

START validateEmail(Paramater: emailToValidate)

    DECLARE passed IS BOOLEAN
    DECLARE localPart IS STRING

```

```

DECLARE domain IS STRING
DECLARE splitEmail IS STRING[]
SET passed = TRUE

IF emailToValidate contains('@@') THEN
    SET splitEmail = emailToValidate.split('@')
    SET localPart = splitEmail[0]
    SET domain = splitEmail[1]

    IF localPart length > 0 AND domain length > 0 THEN
        IF domain contains('.') == FALSE THEN
            passed = FALSE
        ENDIF
    ELSE THEN
        passed = FALSE
    ENDIF
ELSE THEN
    passed = FALSE
ENDIF

RETURN passed
END

START validateDate(Parameter: dateToValidate)
    {date should be format yyyy-mm-dd}
    DECLARE passed IS BOOLEAN
    DECLARE splitDate IS STRING[]

    DECLARE yearLength IS INTEGER
    DECLARE monthLength IS INTEGER
    DECLARE dayLength is INTEGER

    SET passed = TRUE
    IF dateToValidate.length == 11 THEN
        SET splitDate = dateToValidate.length
        IF splitDate.length == 3 THEN
            SET yearLength = splitDate[0].length
            SET monthLength = splitDate[1].length
            SET dayLength = splitDate[2].length

            IF yearLength == 4 AND monthLength == 2 AND dayLength = 2 THEN
                IF !(0 <= monthLength <= 12 AND 0 <= dayLength <= 31)
                    THEN
                        passed = FALSE
                    ENDIF
                ELSE THEN
                    passed = FALSE
                ENDIF
            ELSE THEN
                SET passed = FALSE
            ENDIF
        ELSE THEN
            SET passed = FALSE
        ENDIF
    RETURN passed
END

```

```

START validatePhoneNumber(Paramater: numberToValidate)
  DECLARE passed IS BOOLEAN
  IF numberToValidate.length == 11 THEN
    passed = TRUE
  ELSE
    passed = FALSE
  ENDIF
  RETURN passed
END

```

// Objective 17 Addresses should be looked up via the google maps API to autofill information

// Objective 18 Passwords should have double entry verification

```

START validatePassword()

  DECLARE pass AS BOOLEAN

  INPUT password1FromField {The password entered in the first password field}
  INPUT password2FromField {The password entered in the second password field}

  IF (password1FromField == password2FromField) THEN
    SET pass = TRUE
  ELSE DO
    SET pass = FALSE
  ENDIF

  RETURN pass
END

```

// Objective 19 All fields should have a presence check

```

START presenceCheck() {method of class TextField}
  STRING ATTRIBUTE field IS PREDECLARED IN TEXTFIELD class
  DECLARE passed IS BOOLEAN
  SET passed = TRUE
  IF field == "" THEN
    SET passed = FALSE
  ENDIF
  RETURN passed
END

START presenceCheck() {method of class PasswordField}
  STRING ATTRIBUTE password1 IS PREDECLARED IN PASSWORDFIELDS class
  STRING ATTRIBUTE password2 IS PREDECLARED IN PASSWORDFIELDS class
  DECLARE passed IS BOOLEAN
  SET passed = TRUE
  IF password1 == "" OR password2 == "" THEN
    SET passed = FALSE
  ENDIF
  RETURN passed
END

```

```

START RadioButtonsPresenceCheck() {method of class RadioButtons}
    STRING ATTRIBUTE selectedOption IS PREDECLARED IN RADIOBUTTONS class
    DECLARE passed IS BOOLEAN
    SET passed = TRUE
    IF selected == NULL THEN
        SET passed = FALSE
    ENDIF
    RETURN passed
END

START presenceCheck() {method of class CalendarEntry}
    STRING ATTRIBUTE dateSelected IS PREDECLARED IN CALENDARENTRY class
    DECLARE passed IS BOOLEAN
    SET passed = TRUE
    IF dateSelected == NULL THEN
        SET passed = FALSE
    ENDIF
    RETURN passed
END

START presenceCheck() {method of class FileChooser}
    STRING ATTRIBUTE selectedFilePath IS PREDECLARED IN DocumentUploadField class
    {The path of the file that is selected}
    DECLARE passed IS BOOLEAN
    SET passed = TRUE
    IF selectedFilePath == NULL THEN
        SET passed = FALSE
    ENDIF
    RETURN passed
END

START presenceCheck() {method of class ComboBox}
    STRING ATTRIBUTE selectedOption IS PREDECLARED IN ComboBox class
    DECLARE passed IS BOOLEAN
    SET passed = TRUE
    IF selectedOption == "Please select an option" THEN
        SET passed = FALSE
    ENDIF
    RETURN passed
END

```

// Objective 20 Document upload fields should only allow certain types of files to be uploaded

```

START validateFileChooser(Parameter: typeAllowed)
    DECLARE fileExtension IS STRING
    DECLARE pass IS BOOLEAN

    INPUT filePath {The path of the file selected with the chooser}

    SET pass = FALSE
    SET fileExtension = filePath.split(".")[1]

    IF (typeAllowed == "IMAGES") THEN
        IF (fileExtension == "jpg" OR fileExtension == "gif" OR fileExtension == "png") THEN

```

```

        SET pass = TRUE
    ENDIF
ELSE IF (typeAllowed == "DOCUMENTS") THEN
    IF (fileExtension == "pdf" OR fileExtension == "doc" OR fileExtension ==
"docx") THEN
        SET pass = TRUE
    ENDIF
ELSE IF (typeAllowed == "VIDEOS") THEN
    IF (fileExtension == "mp4" OR fileExtension == "mov" OR fileExtension ==
"avi") THEN
        SET pass = TRUE
    ENDIF
ELSE IF (typeAllowed == "ALL") THEN
    SET pass = TRUE
ENDIF
RETURN PASS
END

```

// Objective 21 Ability to export a Question to file

```

START exportQuestion(Parameter: questionToExport)
    STRING ATTRIBUTE id IS PREDECLARED IN QUESTION class
    STRING ATTRIBUTE difficulty IS PREDECLARED IN QUESTION class
    STRING ATTRIBUTE type IS PREDECLARED IN QUESTION class

    STRING ATTRIBUTE questionID IS PREDECLARED IN QUESTIONPANEL class
    STRING[] ATTRIBUTE components IS PREDECLARED IN QUESTIONPANEL class

    DECLARE questionPanelToExport IS QUESTIONPANEL
    DECLARE i IS INTEGER
    DECLARE questionComponents IS COMPONENT[]

    OUTPUT questionToExport.id + "|" questionToExport.difficulty + "|" +
questionToExport.type

    SET questionPanelToExport = getQuestionPanel(questionToExport) {predefined method
that gets the question panel that corresponds with the question}

    SET questionComponents = questionPanelToExport.components

    FOR i=0 to i=questionComponents.length DO
        OUTPUT questionComponents[i].save()
    ENDFOR
END

```

// Objective 22 Ability to import a Question from file

```

START importQuestion(Parameter: questionToImport)
    STRING ATTRIBUTE id IS PREDECLARED IN QUESTION class
    STRING ATTRIBUTE difficulty IS PREDECLARED IN QUESTION class
    STRING ATTRIBUTE type IS PREDECLARED IN QUESTION class

    STRING ATTRIBUTE questionID IS PREDECLARED IN QUESTIONPANEL class
    STRING[] ATTRIBUTE components IS PREDECLARED IN QUESTIONPANEL class

```

```

DECLARE loadedQuestion IS NEW Question()
DECLARE loadedQuestionPanel IS NEW QuestionPanel()
DECLARE attributeStringArray IS STRING[]
DECLARE componentArray IS Component[]

DECLARE loadedData IS STRING[]

DECLARE i IS INTEGER

{questionToImport is the text from the file}
{The method .getLines() reads the next lines from the file and returns it as a string array}

SET loadedData = questionToImport.getLines()

SET attributeStringArray =loadedData[0].split(" ")

SET loadedQuestion.id = attributeStringArray[0]
SET loadedQuestion.difficulty = attributeStringArray[1]
SET loadedQuestion.type = attributeStringArray[2]

SET componentArray = Component[loadedData.length - 1] {Enough room to store all of
the components}

FOR i=1 TO i=loadedData.length DO
    DECLARE componentType IS STRING
    DECLARE splitData IS STRING[]
    SET splitData = loadedData[i].split(":")
    SET componentType = splitData[0]

    DECLARE componentAttributes IS STRING[]

    SET componentAttributes = splitData[1].split(" ")
    IF componentType == "label" DO
        componentArray[i-1] = loadLabel(componentAttributes) {loadLabel is
        predeclared in the Label objective}
    ELSE componentType == "textfield" DO
        componentArray[i-1] = loadTextField(componentAttributes)
        {loadTextField is predeclared in the TextField objective}
    ELSE componentType == "radiobuttons" DO
        componentArray[i-1] = loadRadioButtons(componentAttributes)
        {loadRadioButtons is predeclared in the RadioButtons objective}
    ELSE componentType == "combobox" DO
        componentArray[i-1] = loadComboBox(componentAttributes)
        {loadComboBox is predeclared in the ComboBox objective}
    ELSE componentType == "checkboxes" DO
        componentArray[i-1] = loadCheckBoxes(componentAttributes)
        {loadCheckBoxes is predeclared in the CheckBoxes objective}
    ELSE componentType == "passwordfields" DO
        componentArray[i-1] = loadPasswordFields(componentAttributes)
        {loadPasswordFields is predeclared in the PasswordFields objective}
    ELSE componentType == "calendarentry" DO
        componentArray[i-1] = loadCalendarEntry(componentAttributes)
        {loadCalendarEntry is predeclared in the CalendarEntry objective}
    ELSE componentType == "documentuploadfield" DO
        componentArray[i-1] = loadDocumentUploadField(componentAttributes)
        {loadDocumentUploadField is predeclared in the DocumentUploadField
        objective}

```

```

        ENDIF
    ENDFOR

    SET loadedQuestionPanel.questionID = loadedQuestion.id
    SET loadedQuestionPanel.components = componentArray

    RETURN loadedQuestion
END

// Objective 23 Ability to attempt a question

// Objective 24 Ability to submit a question
{See submit a form code}

// Objective 25 Ability to create forms
START

    GLOBAL ARRAY OF TYPE FORM formArray IS PREDECLARED {has been filled with
    forms from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextFormLocation IS PREDECLARED {Next free
    location in the array}

    DECLARE newForm IS NEW Form()
    STRING ATTRIBUTE id IS PREDECLARED IN FORM class
    STRING[] ATTRIBUTE questions IS PREDECLARED IN FORM class
    STRING ATTRIBUTE description IS PREDECLARED IN FORM class
    STRING[] ATTRIBUTE mainSkillsTested IS PREDECLARED IN FORM class
    STRING ATTRIBUTE difficulty IS PREDECLARED IN FORM class

    DECLARE newId IS STRING

    DECLARE moreQuestions IS BOOLEAN
    DECLARE i IS INTEGER

    DECLARE newQuestions[100] IS STRING ARRAY
    DECLARE newMainSkillsTested[100] IS STRING ARRAY

    SET newId = CALL METHOD getFreeID() {getFreeID() is predeclared}

    INPUT newDescription
    INPUT newDifficulty

    SET moreQuestions = TRUE
    SET i = 0

    WHILE moreQuestions DO
        INPUT newQuestions[i]
        SET newMainSkillsTested[i] = newQuestions[i].mainSkillTested
        OUTPUT "Continue entering questions? "
        INPUT response
        IF response == "n" THEN {If they entered n (for no)}
            SET moreQuestions = FALSE
        ENDIF
        SET i = i + 1
    
```



```

ENDWHILE

DECLARE trimmedQuestions[i] IS STRING ARRAY
DECLARE trimmedSkills[i] IS STRING ARRAY
DECLARE j IS INTEGER
FOR j=0 TO i DO
    trimmedQuestions[j] = newQuestions[j]
    trimmedSkills[j] = newMainSkillsTested[j]

ENDFOR

SET newForm.id = newId
SET newForm.questions = trimmedQuestions
SET newForm.description = newDescription
SET newForm.mainSkillsTested = trimmedSkills
SET newForm.difficulty = newDifficulty

SET formArray[nextFormLocation] = newForm

CALL METHOD writeDatabase()
END

```

// Objective 26 Ability to create forms from user weaknesses

START createFormFromWeaknesses

```

GLOBAL VARIABLE OF TYPE USER currentUser IS PREDECLARED
GLOBAL VARIABLE OF TYPE QUESTIONSTAT[] questionStatArray IS PREDECLARED
GLOBAL VARIABLE OF TYPE INTEGER nextQuestionStatLocation IS PREDECLARED
{Stores the next free index in the QuestionStat array}
GLOBAL VARIABLE OF TYPE QUESTIONLIST questionList IS PREDECLARED

DECLARE form IS FORM
DECLARE questionsToSort IS QUESTIONSTAT[nextQuestionStatLocation]
DECLARE i IS INTEGER
DECLARE swapped IS BOOLEAN
DECLARE temp IS QUESTIONSTAT
DECLARE questions IS QUESTION[5]

FOR i=0 TO nextQuestionStatLocation
    questionsToSort[i] = questionStatArray[i]
ENDFOR

SET swapped = TRUE

WHILE swapped DO {Bubble sort, descending sort}
    SET swapped = FALSE

    FOR i=0 TO questionsToSort.length DO
        IF questionsToSort[i].averageNumberOfAttemptsNeededToCorrect <
            questionsToSort[i+1].averageNumberOfAttemptsNeededToCorrect THEN
            SET temp = questionsToSort[i+1]
            SET questionsToSort[i+1] = questionsToSort[i]
            SET questionsToSort[i] = temp

            SET swapped = TRUE
        ENDIF
    ENDFOR
ENDWHILE

```

```

                ENDIF
            ENDFOR
        ENDWHILE

        {Select top 5 questions}
        FOR i=0 TO 4 DO
            questions[i] = questionToSort[i]
        ENDFOR

        SET form.questions = questions

        RETURN form
    END

```

// Objective 27 Ability to sort forms by difficulty

```

START
    GLOBAL ARRAY OF TYPE FORM formArray IS PREDECLARED {has been filled with
forms from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextFormLocation IS PREDECLARED {Next free
location in the array}
    DECLARE i IS INTEGER
    DECLARE tempForm IS FORM
    DECLARE swapped IS BOOLEAN
    SET swapped = TRUE
    WHILE swapped == TRUE DO
        SET swapped = FALSE
        FOR i=0 TO nextFormLocation-1 DO
            IF (formArray[i].difficulty > formArray[i+1].difficulty) THEN
                SET tempForm = formArray[i+1]
                SET formArray[i+1] = formArray[i]
                SET formArray[i] = tempForm

                SET swapped = TRUE
            ENDIF
        ENDFOR
    ENDWHILE
END

```

// Objective 28 Ability to filter forms by difficulty

```

START filterByDifficulty(Parameter: filterDifficulty)

    GLOBAL ARRAY OF TYPE FORM formArray IS PREDECLARED {has been filled with
forms from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextFormLocation IS PREDECLARED {Next free
location in the array}
    DECLARE i IS INTEGER
    DECLARE results[nextFormLocation] IS FORM[]
    DECLARE nextResultLocation IS INTEGER

    SET nextResultLocation = 0

    FOR i=0 TO nextFormLocation DO
        IF (formArray[i].difficulty == filterDifficulty) THEN
            SET results[nextResultLocation] = formArray[i]

```

```

        SET nextResultLocation = nextResultLocation + 1
    ENDIF
ENDFOR

DECLARE trimmedResults[nextResultLocation] IS FORM[]

FOR i=0 TO nextResultLocation DO
    SET trimmedResults[i] = results[i]
ENDFOR

RETURN trimmedResults
END

```

// Objective 29 Ability to filter forms by types of questions tested

START filterByType(Parameter: filterTypes)

```

    GLOBAL ARRAY OF TYPE FORM formArray IS PREDECLARED {has been filled with
    forms from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextFormLocation IS PREDECLARED {Next free
location in the array}
    DECLARE i IS INTEGER
    DECLARE j is INTEGER
    DECLARE k is INTEGER
    DECLARE results[nextFormLocation] IS FORM[]
    DECLARE nextResultLocation IS INTEGER
    DECLARE matches is INTEGER

    SET nextResultLocation = 0

    FOR i=0 TO nextFormLocation DO
        SET matches = 0
        FOR j=0 TO filterTypes.length DO
            FOR k=0 TO formArray[i].types.length DO
                IF (filterTypes[j] = formArray[i].types[k]) THEN
                    SET matches = matches + 1
                    BREAK {Move onto the next filter type}
                ENDIF
            ENDFOR
        ENDFOR

        IF (matches == filterTypes.length) THEN {if there was a match for every type in
the filter}
            SET results[nextResultLocation] = formArray[i]
            SET nextResultLocation = nextResultLocation + 1
        ENDIF
    ENDFOR

    DECLARE trimmedResults[nextResultLocation] IS FORM[]

    FOR i=0 TO nextResultLocation DO
        SET trimmedResults[i] = results[i]
    ENDFOR

    RETURN trimmedResults
END

```

// Objective 30 Ability to edit forms

```
START editUser(Parameter: formToEdit)
  STRING ATTRIBUTE id IS PREDECLARED IN FORM class
  STRING[] ATTRIBUTE questions IS PREDECLARED IN FORM class
  STRING ATTRIBUTE description IS PREDECLARED IN FORM class
  STRING[] ATTRIBUTE mainSkillsTested IS PREDECLARED IN FORM class
  STRING ATTRIBUTE difficulty IS PREDECLARED IN FORM class

  DECLARE moreQuestions IS BOOLEAN
  DECLARE i as INTEGER
  DECLARE newQuestions [100] IS STRING ARRAY
  DECLARE newMainSkillsTested[100] IS STRING ARRAY

  OUTPUT formToEdit.id
  INPUT response
  IF response != "" THEN {If they entered a new username}
    SET formToEdit.id = response
  ENDIF

  OUTPUT formToEdit.description
  INPUT response
  IF response != "" THEN
    SET formToEdit.description = response
  ENDIF

  OUTPUT formToEdit.difficulty
  INPUT response
  IF response != "" THEN
    SET formToEdit.difficulty = response
  ENDIF

  OUTPUT formToEdit.questions + " enter 'y' to recreate the list"
  INPUT response
  IF response == 'y' THEN

    SET moreQuestions = TRUE
    SET i = 0
    WHILE moreQuestions DO
      INPUT newQuestions[i]
      SET newMainSkillsTested[i] = newQuestions[i].mainSkillTested
      OUTPUT "Continue entering questions? "
      INPUT response
      IF response == "n" THEN {If they entered n (for no)}
        SET moreQuestions = FALSE
      ENDIF
      SET i = i + 1
    ENDWHILE

    DECLARE trimmedQuestions[i] IS STRING ARRAY
    DECLARE trimmedSkills[i] IS STRING ARRAY
    DECLARE j is INTEGER
    FOR j=0 TO i DO
      trimmedQuestions[j] = newQuestions[j]
      trimmedSkills[j] = newMainSkillsTested[j]
```

```

        ENDFOR
        SET formToEdit.questions= trimmedQuestions
        SET formToEdit.mainSkillsTested = trimmedSkills
    ENDIF

    CALL METHOD writeDatabase()
END

```

// Objective 31 Ability to delete forms

```

START deleteForm(Parameter: formIDToDelete)
    GLOBAL ARRAY OF TYPE FORM formArray IS PREDECLARED {has been filled with
    forms from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextFormLocation IS PREDECLARED {Next free
location in the array}
    DECLARE newArray[formArray.length] IS FORM[]
    DECLARE i IS INTEGER
    DECLARE j IS INTEGER

    SET j = 0

    FOR i=0 TO nextFormLocation DO
        IF (formArray[i].ID != formIDToDelete) THEN {If the form isn't the one that is to
        be deleted}
            SET newArray[j] = formArray[i]
            SET j = j + 1
        ENDIF
    ENDFOR

    SET formArray = newArray
    SET nextFormLocation = nextFormLocation - 1

    CALL METHOD writeDatabase()
END

```

// Objective 32 Ability to export forms to file

```

START exportForm(Parameter: formToExport)

    STRING ATTRIBUTE id IS PREDECLARED IN FORM class
    QUESTION[] ATTRIBUTE questions IS PREDECLARED IN FORM class
    STRING ATTRIBUTE description IS PREDECLARED IN FORM class
    STRING[] ATTRIBUTE mainSkillsTested IS PREDECLARED IN FORM class
    STRING ATTRIBUTE difficulty IS PREDECLARED IN FORM class

    DECLARE i IS INTEGER

    OUTPUT formToExport.id + "|" formToExport.description + "|" +
    formToExport.mainSkillsTested + "|" + formToExport.difficulty

    FOR i=0 TO i=formToExport.questions.length DO
        exportQuestion(formToExport.questions[i]) {exportQuestion is predeclared see
        export question objective}
    ENDFOR
END

```

// Objective 33 Ability to import forms from file

START importForm(Parameter: formToImport)

GLOBAL ARRAY OF TYPE QUESTION questionArray IS PREDECLARED {has been filled with questions from file}

GLOBAL VARIABLE OF TYPE INTEGER nextQuestionLocation IS PREDECLARED {Next free location in the array}

STRING ATTRIBUTE id IS PREDECLARED IN FORM class

QUESTION[] ATTRIBUTE questions IS PREDECLARED IN FORM class

STRING ATTRIBUTE description IS PREDECLARED IN FORM class

STRING[] ATTRIBUTE mainSkillsTested IS PREDECLARED IN FORM class

STRING ATTRIBUTE difficulty IS PREDECLARED IN FORM class

DECLARE loadedForm IS NEW Form()

DECLARE attributeStringArray IS STRING[]

DECLARE loadedQuestions is QUESTION[]

DECLARE loadedData IS STRING[]

DECLARE i IS INTEGER

{formToImport is the text from the file}

{The method .getLines() reads the next lines from the file and returns it as a string array}

SET loadedData = formToImport.getLines()

SET attributeStringArray = loadedData[0].split(" | ")

SET loadedForm.id = attributeStringArray[0]

SET loadedForm.description = attributeStringArray[1]

SET loadedForm.mainSkillsTested = attributeStringArray[2]

SET loadedForm.difficulty = attributeStringArray[3]

SET loadedQuestions = Questions[loadedData.length - 1] {Enough room to store all of the questions}

FOR i=1 TO i=loadedData.length DO

loadedQuestions[i-1] = importQuestion(loadedData[i]) {importQuestion is predeclared in the import questions objective}

ENDFOR

DECLARE j IS INTEGER

DECLARE inSystem is BOOLEAN

FOR i=0 TO loadedQuestions.length DO

SET inSystem = FALSE

FOR j=0 to j=nextQuestionLocation

IF (questionArray[j].id == loadedQuestions[i].id) THEN

SET inSystem = TRUE

ENDIF

ENDFOR

IF (inSystem == FALSE) THEN

SET questionArray[nextQuestionLocation] = loadedQuestions[i]

```

        SET nextQuestionLocation = nextQuestionLocation + 1
    ENDIF
ENDFOR
END

// Objective 34 Ability to attempt a form

// Objective 35 Ability to submit a form

START validateForm()
    GLOBAL VARIABLE OF TYPE FORM formInProgress IS PREDECLARED {The form that the
    user is currently attempting}
    GLOBAL VARIABLE OF TYPE USER currentLoggedInUser IS PREDECLARED

    GLOBAL VARIABLE OF TYPE QUESTIONPANEL[] questions IS PREDECLARED {Contains
    the questions that are in the form}

    GLOBAL VARIABLE OF TYPE INTEGER[] failedValidationChecks IS PREDECLARED
    GLOBAL VARIABLE OF TYPE LONG[] timeToCompleteQuestions IS PREDECLARED

    DECLARE i IS INTEGER
    DECLARE allCorrect IS BOOLEAN
    DECLARE passed IS BOOLEAN

    FOR i=0 TO questions.length DO
        SET passed = questions[i].validate() {validate() is predeclared in each type of
        question class}

        IF !passed THEN
            SET allCorrect = FALSE
            currentLoggedInUser.addFailedValidation(questions[i])
            {addFailedAttempt is predeclared}
            SET failedValidationChecks[i] = failedValidationChecks[i] + 1
        ENDIF

    ENDFOR

    IF allCorrect THEN
        FOR i=0 TO questions.length DO
            IF failedValidationChecks[i] > 0 THEN
                currentLoggedInUser.addNumberOfAttemptsNeededToCorrect(qu
                estions[i], failedValidationChecks[i]) {addNumberOfAttempts is
                predeclared}
            ENDIF
            currentLoggedInUser.addTimeTakenToComplete(questions[i],
            timeToCompleteQuestions[i]) {addTimeTakenToComplete is predeclared}
        ENDFOR

        OUTPUT "Form Complete"
    ENDIF
END

// Objective 36 The time taken to complete each question should be stored

START addTimeTakenToComplete(Parameter: question, timeTakenToComplete)

```

```

GLOBAL VARIABLE OF TYPE USER currentLoggedInUser IS PREDECLARED
GLOBAL VARIABLE OF TYPE QUESTIONSTATLIST questionStats IS PREDECLARED
DECLARE questionStat IS QUESTIONSTAT

DECLARE newTimeTakenToComplete IS LONG[]
DECLARE i IS INTEGER

SET questionStat = questionStats.getStat(question) {getStat is predeclared and gets the
questionStat that corresponds with the question}

SET newTimeTakenToComplete = LONG[questionStat.timeTakenToComplete.length]
{This array operates as a queue}

FOR i=1 TO newTimeTakenToComplete.length DO
    SET newTimeTakenToComplete[i] = questionStat.timeTakenToComplete[i-1]
ENDFOR

SET newTimeTakenToComplete[0] = timeTakenToComplete

SET questionStat.timeTakenToComplete = newTimeTakenToComplete {Overwrite the old
array}

END

```

// Objective 37 The number of times that they've failed the validation check for a question should be stored

```

START addFailedValidation(Parameter: question)
    GLOBAL VARIABLE OF TYPE USER currentLoggedInUser IS PREDECLARED
    GLOBAL VARIABLE OF TYPE QUESTIONSTATLIST questionStats IS PREDECLARED
    DECLARE questionStat IS QUESTIONSTAT

    SET questionStat = questionStats.getStat(question) {getStat is predeclared and gets the
    questionStat that corresponds with the question}

    SET questionStat.timesFailedValidation = questionStat.timesFailedValidation + 1
END

START addNumberOfAttemptsNeededToCorrect(Parameter: question,
numberOfAttemptsNeededToCorrect)

    GLOBAL VARIABLE OF TYPE USER currentLoggedInUser IS PREDECLARED
    GLOBAL VARIABLE OF TYPE QUESTIONSTATLIST questionStats IS PREDECLARED
    DECLARE questionStat IS QUESTIONSTAT

    DECLARE newNumberOfAttemptsNeededToCorrect IS INTEGER[]
    DECLARE i IS INTEGER

    SET questionStat = questionStats.getStat(question) {getStat is predeclared and gets the
    questionStat that corresponds with the question}

    SET newNumberOfAttemptsNeededToCorrect =
    LONG[questionStat.numberOfAttemptsNeededToCorrect.length] {This array operates as
    a queue}

```



```

FOR i=1 TO newNumberOfAttemptsNeededToCorrect.length DO
    SET newNumberOfAttemptsNeededToCorrect[i] =
        questionStat.numberOfAttemptsNeededToCorrect[i-1]
ENDFOR

SET newNumberOfAttemptsNeededToCorrect[0] = numberOfAttemptsNeededToCorrect

SET questionStat.numberOfAttemptsNeededToCorrect =
    newNumberOfAttemptsNeededToCorrect {Overwrite the old array}

END

```

// Objective 38 Ability to add users

START

```

GLOBAL ARRAY OF TYPE USER[] userArray IS PREDECLARED {has been filled with
users from file}
GLOBAL VARIABLE OF TYPE INTEGER nextUserLocation IS PREDECLARED {Next free
location in the array}

```

```

DECLARE newUser IS NEW User()
STRING ATTRIBUTE id IS PREDECLARED IN USER class
STRING ATTRIBUTE username IS PREDECLARED IN USER class
STRING ATTRIBUTE password IS PREDECLARED IN USER class
STRING ATTRIBUTE firstName IS PREDECLARED IN USER class
STRING ATTRIBUTE lastName IS PREDECLARED IN USER class
STRING ATTRIBUTE dateOfBirth IS PREDECLARED IN USER class
STRING ATTRIBUTE phoneNumber IS PREDECLARED IN USER class

```

```

STRING ARRAY ATTRIBUTE sessionsPresentAt IS PREDECLARED IN USER class
DECLARE newId IS STRING
DECLARE moreSessions IS BOOLEAN
DECLARE i as INTEGER
DECLARE newSessions[100] IS STRING ARRAY

```

```

SET newId = CALL METHOD getFreeID() {getFreeID() is predeclared}
INPUT newUsername
INPUT newPassword
INPUT newFirstName
INPUT newLastName
INPUT newDateOfBirth
INPUT newPhoneNumber

```

```

SET moreSessions = TRUE
SET i = 0
WHILE moreSessions DO
    INPUT newSessions[i]

    OUTPUT "Continue entering sessions? "
    INPUT response
    IF response == "n" THEN {If they entered n (for no)}
        SET moreSessions = FALSE
    ENDIF
    SET i = i + 1
ENDWHILE

```

```

DECLARE trimmedSessions[i] IS STRING ARRAY

DECLARE j is INTEGER
FOR j=0 TO i DO
    trimmedSessions[j] = newSessions[j]
ENDFOR

SET newUser.id = newId
SET newUser.username = newUsername
SET newUser.password = newPassword
SET newUser.firstName = newFirstName
SET newUser.lastName = newLastName
SET newUser.dateOfBirth = newDateOfBirth
SET newUser.phoneNumber = newPhoneNumber
SET newUser.sessionsPresentAt = trimmedSessions

SET userArray[nextUserLocation] = newUser

CALL METHOD writeDatabase()
END

```

// Objective 39 Ability to edit users

```

START editUser(Parameter: userToEdit)
    STRING ATTRIBUTE id IS PREDECLARED IN USER class
    STRING ATTRIBUTE username IS PREDECLARED IN USER class
    STRING ATTRIBUTE password IS PREDECLARED IN USER class
    STRING ATTRIBUTE firstName IS PREDECLARED IN USER class
    STRING ATTRIBUTE lastName IS PREDECLARED IN USER class
    STRING ATTRIBUTE dateOfBirth IS PREDECLARED IN USER class
    STRING ATTRIBUTE phoneNumber IS PREDECLARED IN USER class

    STRING ARRAY ATTRIBUTE sessionsPresentAt is PREDECLARED IN USER class

    DECLARE moreSessions IS BOOLEAN
    DECLARE i as INTEGER
    DECLARE newSessions[100] IS STRING ARRAY

    OUTPUT userToEdit.username
    INPUT response
    IF response != "" THEN {If they entered a new username}
        SET userToEdit.username = response
    ENDIF

    OUTPUT userToEdit.password
    INPUT response
    IF response != "" THEN
        SET userToEdit.password = response
    ENDIF

    OUTPUT userToEdit.firstName
    INPUT response
    IF response != "" THEN
        SET userToEdit.firstName= response
    ENDIF

```

```

OUTPUT userToEdit.lastName
INPUT response
IF response != "" THEN
    SET userToEdit.lastName = response
ENDIF

OUTPUT userToEdit.dateOfBirth
INPUT response
IF response != "" THEN
    SET userToEdit.dateOfBirth = response
ENDIF

OUTPUT userToEdit.phoneNumber
INPUT response
IF response != "" THEN
    SET userToEdit.phoneNumber = response
ENDIF

OUTPUT userToEdit.sessionsPresentAt + " enter 'y' to recreate the list"
INPUT response
IF response == 'y' THEN

    SET moreSessions = TRUE
    SET i = 0
    WHILE moreSessions DO
        INPUT newSessions[i]

        OUTPUT "Continue entering sessions? "
        INPUT response
        IF response == "n" THEN {If they entered n (for no)}
            SET moreSessions = FALSE
        ENDIF
        SET i = i + 1
    ENDWHILE

    DECLARE trimmedSessions[i] IS STRING ARRAY

    DECLARE j is INTEGER
    FOR j=0 TO i DO
        trimmedSessions[j] = newSessions[j]
    ENDFOR

    SET userToEdit.sessionsPresentAt = trimmedSessions
ENDIF

CALL METHOD writeDatabase()
END

```

// Objective 40 Ability to delete users

```

START deleteUser(Parameter: userIDToDelete)
    GLOBAL ARRAY OF TYPE USER userArray IS PREDECLARED {has been filled with users
    from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextUserLocation IS PREDECLARED {Next free
    location in the array}
    DECLARE newArray[userArray.length] IS USER[]

```

```

DECLARE i IS INTEGER
DECLARE j IS INTEGER

SET j = 0

FOR i=0 TO nextUserLocation DO
    IF (userArray[i].ID != userIDToDelete) THEN {If the user isn't the one that is to be deleted}
        SET newArray[i] = userArray[i]
        SET j = j + 1
    ENDIF
ENDFOR

SET userArray = newArray
SET nextUserLocation = nextUserLocation - 1

CALL METHOD writeDatabase()
END

```

// Objective 41 Ability to search through users

```

START filterByFirstName(Parameter: filterName)

    GLOBAL ARRAY OF TYPE USER userArray IS PREDECLARED {has been filled with users from file}
    GLOBAL VARIABLE OF TYPE INTEGER nextUserLocation IS PREDECLARED {Next free location in the array}
    DECLARE i IS INTEGER
    DECLARE results[nextUserLocation] IS USER[]
    DECLARE nextResultLocation IS INTEGER

    SET nextResultLocation = 0

    FOR i=0 TO nextUserLocation DO
        IF (userArray[i].firstName == filterName) THEN
            SET results[nextResultLocation] = userArray[i]
            SET nextResultLocation = nextResultLocation + 1
        ENDIF
    ENDFOR

    DECLARE trimmedResults[nextResultLocation] IS USER[]

    FOR i=0 TO nextResultLocation DO
        SET trimmedResults[i] = results[i]
    ENDFOR

    RETURN trimmedResults
END

```

// Objective 42 Ability to produce a report on the user's weak areas/progress

```

START produceReport(Parameter: userToReport)
    GLOBAL VARIABLE OF TYPE QUESTIONSTATLIST questionStats IS PREDECLARED

    QUESTIONSTAT[] ATTRIBUTE questionStatArray IS PREDECLARED IN

```

```

QUESTIONSTATLIST class
INTEGER ATTRIBUTE nextQuestionStatLocation IS PREDECLARED IN QUESTIONSTATLIST
class

STRING ATTRIBUTE id IS PREDECLARED IN QUESTIONSTAT class
LONG[] ATTRIBUTE timeTakenToComplete IS PREDECLARED IN QUESTIONSTAT class
INTEGER ATTRIBUTE timesFailedValidation IS PREDECLARED IN QUESTIONSTAT class
LONG[] ATTRIBUTE numberOfAttemptsNeededToCorrect IS PREDECLARED IN
QUESTIONSTAT class

INTEGER ATTRIBUTE nextQuestionStatLocation IS PREDECLARED IN QUESTIONSTATLIST
class

DECLARE questionStat IS QUESTIONSTAT
DECLARE i IS INTEGER

FOR i=0 TO i=questionStats.nextQuestionStatLocation DO
    SET questionStat = questionStats.questionStatArray[i]
    OUTPUT questionStat.id
    OUTPUT questionStat.timeTakenToComplete
    OUTPUT questionStat.timesFailedValidation
    OUTPUT questionStat.numberOfAttemptsNeededToCorrect
ENDFOR
END

```

// Objective 43 How much of each form a user has attempted should be tracked.

```

START getPercentageComplete() {works out what percentage of the form that the user is
attempting the user has completed}
    GLOBAL VARIABLE OF TYPE FORM formInProgress IS PREDECLARED {The form that the
    user is currently attempting}
    GLOBAL VARIABLE OF TYPE QUESTION[] questions IS PREDECLARED {Contains the
    questions that are in the form}

    DECLARE i IS INTEGER
    DECLARE passed IS BOOLEAN
    DECLARE numberCorrect IS INTEGER
    DECLARE percentageCorrect IS INTEGER

    FOR i=0 TO questions.length DO
        SET passed = questions[i].validate() {validate() is predeclared in each type of
        question class}

        IF passed THEN
            SET numberCorrect = numberCorrect + 1
        ENDIF
    ENDFOR

    SET percentageComplete = (numberCorrect/questions.length) * 100

    RETURN percentageComplete
END

```

```

START addPercentageComplete(Parameter: formID, percentageComplete) {method of

```

```

FormInProgressList object}
  GLOBAL VARIABLE OF TYPE USER currentLoggedInUser IS PREDECLARED
  GLOBAL VARIABLE OF TYPE FORMINPROGRESSLIST formsInProgress IS PREDECLARED
  DECLARE form IS FORMINPROGRESS

  SET form = formsInProgress.getForm(formID) {getForm is predeclared and gets the
  formInProgress object that corresponds with the formID}

  SET form.percentageComplete = percentageComplete
END

```

// Objective 44 First name, last name, Date of birth and phone number should be stored for each user

{See add users objective}

// Objective 45 The attendance for each user should be automatically tracked

```

START trackAttendance() {Called when a user logs in is a method of the user class}
  GLOBAL VARIABLE OF TYPE STRING[] sessionsAttended IS PREDECLARED

  DECLARE newSessionsAttended IS STRING[sessionsAttended.length + 1]
  DECLARE currentDate IS STRING

  SET currentDate = getCurrentDate() {getCurrentDate is predeclared and returns the
  current date}

  SET newSessionsAttended[sessionsAttended.length] = currentDate

  SET sessionsAttended = newSessionsAttended
END

```

// Objective 46 The system should determine the reason for a user missing a session

```

START attendedLastSession()
  GLOBAL VARIABLE OF TYPE USER currentLoggedInUser IS PREDECLARED
  DECLARE mostRecentSession IS STRING
  DECLARE lastSessionAttended IS STRING
  DECLARE numberOfSessionsAttended IS INTEGER
  DECLARE pass IS BOOLEAN

  SET mostRecentSession = getMostRecentThursday() {getMostRecentThursday is
  predeclared}

  SET numberOfSessionsAttended = currentLoggedInUser.sessionsAttended.length
  SET lastSessionAttended =
  currentLoggedInUser.sessionsAttended[numberOfSessionsAttended-1]

  IF lastSessionAttended == mostRecentSession THEN
    SET pass = TRUE
  ELSE THEN
    SET pass = FALSE
  ENDIF

```

```

    RETURN pass
END

START
    GLOBAL VARIABLE OF TYPE STRING ARRAY notRunningSessions IS PREDECLARED
    {Stores the dates of the sessions that didn't run}
    GLOBAL VARIABLE OF TYPE INTEGER nextNotRunningSession IS PREDECLARED
    DECLARE mostRecentSession IS STRING

    CALL METHOD login(){login is predeclared and triggers the login process}
    IF (attendedLastSession() == FALSE) THEN
        OUTPUT "What was the reason for missing the last session?"
        OUTPUT "1. Session not running"
        OUTPUT "2. Absent"
        INPUT response
        IF (reponse == 1) THEN
            SET mostRecentSession = getMostRecentThursday()
            SET notRunningSessions[nextNotRunningSession] = mostRecentSession
        ENDIF
    ENDIF
END

```

// Objective 47 Should be simple

// Objective 48 Login page which allows users or admins to login

// Objective 49 There should be a main menu panel

// Objective 50 There should be a panel which will display the forms to the user and allow them to search through them

// Objective 51 Allow user to select a form to be attempted

// Objective 52 Allow form to be submitted

// Objective 53 There should be a panel which will display the questions to the user and allow them to search through them

// Objective 54 Allow user to select a question to be attempted

// Objective 55 Allow question to be submitted

// Objective 56 Questions that have failed their validation check should be highlighted

// Objective 57 The user should be able to create forms by selecting questions from a list

// Objective 58 There should be a statistics screen which shows the user's progress

// Objective 59 There should be a context sensitive help button available in

each window and panel of the application

// Objective 60 The admin user should have a different main menu

// Objective 61 The admin user should be able to view information about each user

// Objective 62 The admin user should be able to edit information about each user

// Objective 63 The admin user should be able to take a register

START

GLOBAL ARRAY OF TYPE USER[] userArray IS PREDECLARED {has been filled with users from file}

GLOBAL VARIABLE OF TYPE INTEGER nextUserLocation IS PREDECLARED {Next free location in the array}

DECLARE i IS INTEGER

DECLARE currentDate IS STRING

SET currentDate = getCurrentDate() {getCurrentDate() is predeclared and returns the current date}

FOR i=0 TO i=nextUserLocation

OUTPUT userArray[i].firstName + " " + userArray[i].lastName

INPUT response

IF response == "y" THEN

userArray[i].addAttendance(currentDate) {addAttendance is a predeclared method of the class user and appends the date (passed as the argument) to the end of the user's sessionsPresentAt array}

ENDIF

ENDFOR

END